IA Générative Multi Modale
Introduction à l'IA Générative, Bases de Pytorch

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



IA Discriminative — *Apprendre à distinguer*

L'IA discriminative apprend la frontière entre les classes (ou la relation entrée → sortie) à partir de données étiquetées. Elle prédit une étiquette ou une valeur pour une nouvelle donnée, mais ne génère pas de nouvelles données.



Chat
Chien
Oiseau
Voiture
...

Données étiquetées
(exemples d'apprentissage)

Données d'entrée (x)

Modèle discriminatif
(ex. CNN, ResNet, SVM, ...)



Apprend P(y | x)

Prédiction
(Classe ou valeur)

Chat	0.92
Chien	0.05
Oiseau	0.02
Voiture	0.01

...

Tâches typiques :
Classification, détection, régression, reconnaissance, ...

VS

IA Générative — *Apprendre à créer*

L'IA générative apprend la distribution des données P(x) (ou P(x | condition)) et est capable de générer de nouvelles données réalistes et cohérentes, telles que des images, du texte, de l'audio, de la vidéo ou du code.

Prompt / Condition (exemple)

- « Un chat portant un nœud papillon, assis dans un salon élégant »
- « A scenic mountain landscape at sunset »
- « Écris un court poème sur l'espoir »
- ...

Modèle génératif
(ex. GAN, VAE, Diffusion, Transformers, ...)



Apprend P(x) ou P(x | condition)

Nouvelle donnée générée



Image générée



Image générée

Dans chaque aube naît une chance
Un rêve qui se lève et avance
L'espoir éclaire même la nuit
Et fait de demain un cœur qui vit

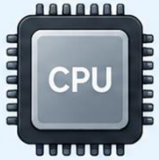
Texte généré

...

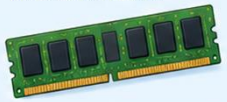
Tâches typiques :
Génération d'images, de texte, d'audio, de vidéo, de code, de données synthétiques, ...

CPU (Hôte)

Traitement général, préparation des données



RAM (mémoire principale)
Stocke les données, le code Python, les tensors sur CPU, etc.



- Exécute le code Python (PyTorch)
- Prépare et prétraite les données
- Gère l'entraînement (boucle, optimisation, ...)
- Peut effectuer les calculs sur CPU (si pas de GPU)

.to("cuda") — Transfert de données — .to("cpu")

GPU (Device)

Calculs massivement parallèles pour les tenseurs



VRAM
(mémoire du GPU)
Stocke les tensors, les paramètres du modèle, les activations, ...



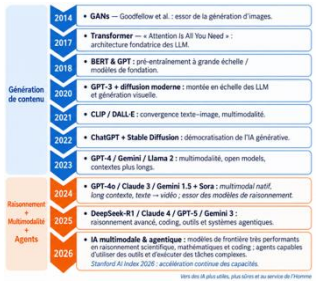
- Exécute les opérations sur les tensors (très parallèles)
- Idéal pour l'entraînement des grands modèles
- Mémoire dédiée : la VRAM (plus rapide que la RAM)
- Nécessite CUDA (NVIDIA) pour être accessible depuis PyTorch

CUDA

Couche logicielle de NVIDIA

PyTorch (torch.cuda)
↓
CUDA Runtime (cuBLAS, cuDNN, ...)
↓
Pilote NVIDIA (Driver)

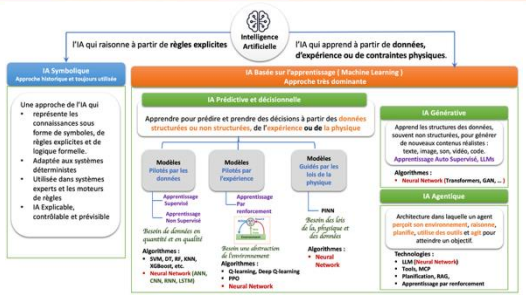
NVIDIA



QWEN-3

Gemma 4





Intelligence Artificielle

l'IA qui raisonne à partir de **règles explicites**

l'IA qui apprend à partir de **données, d'expérience ou de contraintes physiques.**

IA Symbolique
Approche historique et toujours utilisée

Une approche de l'IA qui
- représente les connaissances sous forme de symboles, de règles explicites et de logique formelle.
- Adaptée aux systèmes déterministes
- Utilisée dans systèmes experts et les moteurs de règles
- IA Explicable, contrôlable et prévisible

IA Basée sur l'apprentissage (Machine Learning)
Approche très dominante

IA Prédictive et décisionnelle

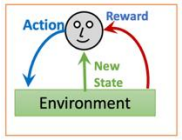
Apprendre pour prédire et prendre des décisions à partir des **données structurées ou non structurées,** de l'**expérience** **ou de** **la physique**

Modèles Pilotés par les données
- Apprentissage Supervisé
- Apprentissage Non Supervisé

Besoin de données en quantité et en qualité

Algorithmes :
- SVM, DT, RF, KNN, XGBoost, etc.
- Neural Network (ANN, CNN, RNN, LSTM)

Modèles Pilotés par l'expérience
- Apprentissage Par renforcement



Besoin une abstraction de l'environnement

Algorithmes :
- Q-learning, Deep Q-learning
- PPO
- Neural Network

Modèles Guidés par les lois de la physique
- PINN

Besoin des lois de la, physique et des données

Algorithmes :
- Neural Network

IA Générative

Apprend les structures des données, souvent non structurées, pour générer de nouveaux contenus réalistes : texte, image, son, vidéo, code.
Apprentissage Auto Supervisé, LLMs

Algorithmes :
- Neural Network (Transformers, GAN, …)

IA Agentique

Architecture dans laquelle un agent **perçoit son environnement**, **raisonne**, **planifie**, **utilise des outils** et **agit** pour atteindre un objectif.

Technologies :
- LLM (Neural Network)
- Tools, MCP
- Planification, RAG,
- Apprentissage par renforcement

Quelques Rappels

IA Basée sur l'apprentissage (Machine Learning)
Approche très dominante

IA Prédictive et décisionnelle

Apprendre pour prédire et prendre des décisions à partir des **données structurées ou non structurées,** de l'**expérience** **ou de** **la physique**

Modèles Pilotés par les données

- Apprentissage Supervisé
- Apprentissage Non Supervisé

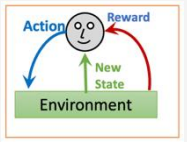
Besoin de données en quantité et en qualité

Algorithmes :

- SVM, DT, RF, KNN, XGBoost, etc.
- Neural Network (ANN, CNN, RNN, LSTM)

Modèles Pilotés par l'expérience

- Apprentissage Par renforcement



Besoin une abstraction de l'environnement

Algorithmes :

- Q-learning, Deep Q-learning
- PPO
- Neural Network

Modèles Guidés par les lois de la physique

- PINN

Besoin des lois de la, physique et des données

Algorithmes :

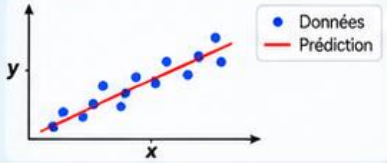
- Neural Network

Les grands paradigmes de l'apprentissage automatique

Différentes façons d'apprendre, selon le type de données et l'objectif

1 Apprentissage supervisé – Régression

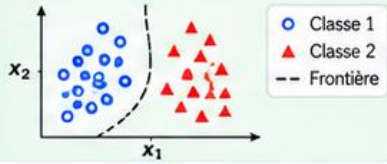
Apprendre une fonction qui prédit une valeur réelle à partir de données étiquetées (x, y).



Exemples
- Prix d'une maison
- Consommation d'énergie
- Prédiction de température
- ...

2 Apprentissage supervisé – Classification

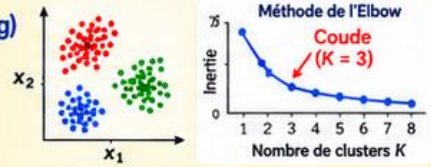
Apprendre à prédire une classe discrète à partir de données étiquetées (x, y).



Exemples
- Détection de spam
- Reconnaissance d'images
- Diagnostic médical
- ...

3 Apprentissage non supervisé (Clustering)

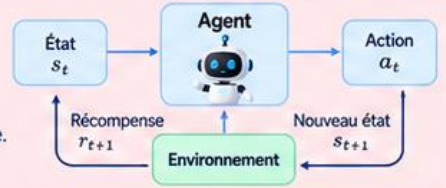
Découvrir une structure cachée dans des données non étiquetées.



Exemples
- Segmentation clients
- Regroupement d'images
- Analyse de documents
- ...

4 Apprentissage par renforcement

Apprendre une politique d'action en interagissant avec un environnement pour maximiser une récompense cumulative.



Exemples
- Jeux (Go, Atari)
- Robotique
- Véhicules autonomes
- Optimisation
- ...

5 Apprentissage auto supervisé

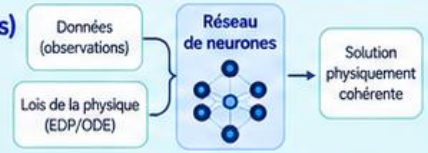
Apprendre des représentations utiles à partir de données non étiquetées en créant soi-même des tâches de prédiction (ex. données masquées, parties manquantes, ...).



Exemples
- Modèles de langage (LLM)
- Représentations d'images
- Audio, vidéo
- Pré-entraînement
- ...

6 Réseaux informés par la physique (PINNs)

Combiner les données et les lois physiques (équations différentielles) pour apprendre des solutions cohérentes et robustes.



Exemples
- Mécanique des fluides
- Transfert de chaleur
- Dynamique de systèmes
- Finance, santé, etc.

Un même objectif : apprendre à partir des données, sous différentes formes et contraintes, pour comprendre, prédire et agir dans le monde réel.

IA Discriminative vs IA Générative

Comprendre le monde vs Créer de nouveaux mondes

IA Discriminative — *Apprendre à distinguer*

L'IA discriminative apprend la frontière entre les classes (ou la relation entrée → sortie) à partir de données étiquetées. Elle prédit une étiquette ou une valeur pour une nouvelle donnée, mais ne génère pas de nouvelles données.


Chat


Chien


Oiseau

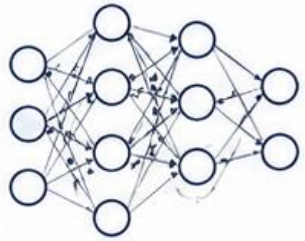

Voiture

...

Données étiquetées
(exemples d'apprentissage)

Données d'entrée (x)

Modèle discriminatif
(ex. CNN, ResNet, SVM, ...)



Apprend P(y | x)

Prédiction
(Classe ou valeur)

Chat	0.92
Chien	0.05
Oiseau	0.02
Voiture	0.01

...

Tâches typiques :
Classification, détection, régression, reconnaissance, ...

VS

IA Générative — *Apprendre à créer*

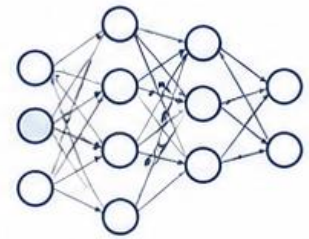
L'IA générative apprend la distribution des données P(x) (ou P(x | condition)) et est capable de générer de nouvelles données réalistes et cohérentes, telles que des images, du texte, de l'audio, de la vidéo ou du code.

Prompt / Condition (exemple)

- « Un chat portant un nœud papillon, assis dans un salon élégant »
- « A scenic mountain landscape at sunset »
- « Écris un court poème sur l'espoir »

...

Modèle génératif
(ex. GAN, VAE, Diffusion, Transformers, ...)



Apprend P(x) ou P(x | condition)

Nouvelle donnée générée


Image générée


Image générée

Dans chaque aube naît une chance
Un rêve qui se lève et avance
L'espoir éclaire même la nuit
Et fait de demain un cœur qui vit

Texte généré

...

Tâches typiques :
Génération d'images, de texte, d'audio, de vidéo, de code, de données synthétiques, ...

Objectif	Type de données	Exemples de modèles	Applications
Prédire une étiquette ou une valeur en apprenant P(y | x).	Données étiquetées.	CNN, ResNet, SVM, XGBoost, ...	Classification d'images, détection d'objets, analyse de sentiments, diagnostic médical, ...

Objectif	Type de données	Exemples de modèles	Applications
Générer de nouvelles données en apprenant P(x) ou P(x | condition).	Données non étiquetées ou conditionnées (prompt, texte, ...).	GAN, VAE, Diffusion, Transformers (GPT, DALL·E, Stable Diffusion, ...)	Création d'images, texte, vidéo, audio, code, données synthétiques, ...

L'IA discriminative comprend le monde existant. L'IA générative imagine de nouveaux mondes.

Deux approches complémentaires pour une IA plus puissante.

Generative AI (State Of Art)

- Generative AI refers to a type of Artificial Intelligence that is capable of generating new data, such as images, music, or text, based on a set of training data.
- Reasons that have led to a significant acceleration in the pace of research and development in generative AI.
  - **Powerful Language Models** :
    - For NLP : (GPT, BERT) : Machine translation, Text summarization, Sentiment analysis, Conversational AI
    - For Computer Vision : GANs, Stable Diffusion, DALL-E, Midjourney
  - **Abundant Data**:
    - In 2022, users sent around **650 million Tweets** per **day**.
    - In 2022, **333.2 billion emails** were sent every **day**.
  - **Hardware Advancements** : **GPU & TPU , Cloud Computing**

https://thispersondoesnotexist.com/

2014
- Generative Adversarial Networks (GANs) (Ian Goodfellow)

2017
- Break through in NLP : **Transformers** (google)

Attention is All You need (https://arxiv.org/pdf/1706.03762.pdf)

2018
- Advent of Large Models :
  - **BERT** (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) , Google
  - **GPT** (**G**enerative **P**re-trained **T**ransformers **) (Open AI)**

2019
- Advanced GANs – Style GANs

2020
- **GPT-3** (Premier LLM)(Open AI)

2021
- Vision Transformers – CLIP, **DALL-E** : Images (OpenAI)

2022
- Chat GPT : **GPT – 3.5 (Text)**, **DALL-E 2(Images), Stable Diffusion**

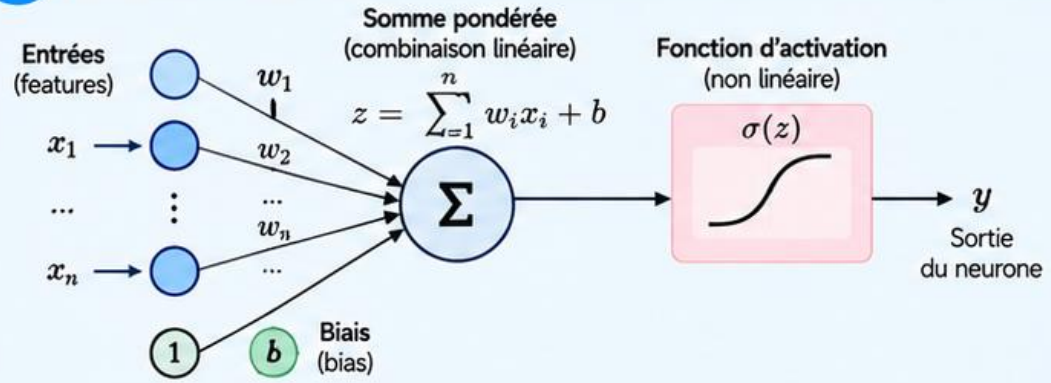
.. 26
- GPT-4o, GPT5, GPT6, Claude 3, Claude 4, GMINI 3, LIAMMA, ….
- IA Générative Multimodale et Agentique,

Réseaux de neurones artificiels : du neurone à l'apprentissage

Apprendre une fonction à partir des données en ajustant automatiquement les poids

Un réseau de neurones apprend en minimisant une erreur sur des données grâce à l'ajustement de ses poids.

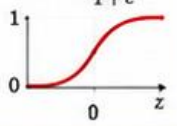
1 Le neurone artificiel



Exemples de fonctions d'activation

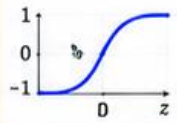
Sigmoïde

$\sigma(z) = \frac{1}{1+e^{-z}}$



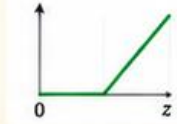
Tanh

$\tanh(z)$



ReLU

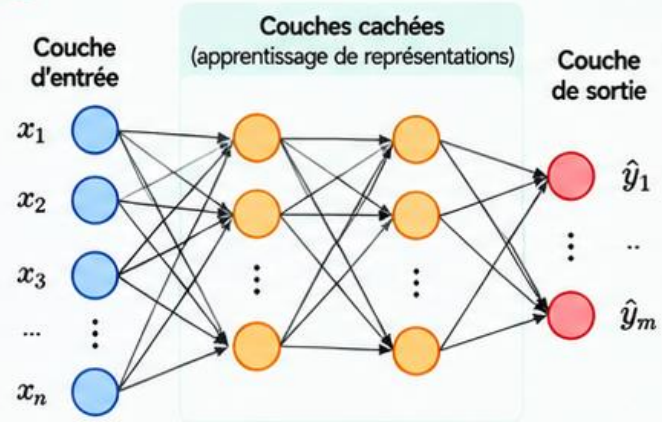
$\text{ReLU}(z) = \max(0, z)$



Rôle des paramètres

- w_i : poids (importance de chaque entrée)
- b : biais (décalage)
- σ : fonction d'activation (introduit la non-linéarité)
- z : combinaison linéaire
- y : sortie du neurone

2 Du neurone au réseau de neurones



Chaque couche calcule :

$$h = \sigma(Wx + b)$$

(où x peut être la sortie de la couche précédente)

3 La fonction de perte (Loss)

Mesure l'écart entre la prédiction $\hat{y}$ et la valeur réelle y.

Régression (ex. MSE)

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Classification (ex. entropie croisée)

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik})$$

Objectif : trouver les poids W et biais b qui minimisent $\mathcal{L}$

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

$(\theta = \{W, b\})$

4 Apprentissage : descente de gradient

Données d'entraînement $\{(x_i, y_i)\}$

→ **Passage avant (Forward)** : Calcul des prédictions $\hat{y} = f(x;\theta)$

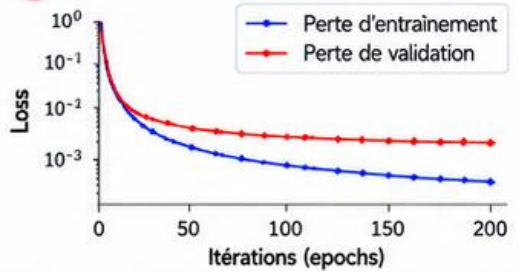
→ **Calcul de la perte (Loss)** : $\mathcal{L}(\theta)$

→ **Rétropropagation (Backward)** : Calcul des gradients $\nabla_\theta \mathcal{L}$

→ **Mise à jour des paramètres** : $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ (η : taux d'apprentissage)

Répéter sur plusieurs itérations (epochs)

5 Évolution de la perte



6 Utilisation (inférence)

Nouvelle donnée x → **Modèle entraîné** $f(x;\theta^*)$ → Prédiction $\hat{y}$

À retenir :

- ✓ Un neurone effectue une somme pondérée suivie d'une fonction d'activation.
- ✓ Un **réseau de neurones** empile plusieurs couches pour apprendre des représentations complexes.
- ✓ Les poids et biais sont appris en minimisant une fonction de perte via la descente de gradient.
- ✓ Une fois entraîné, le modèle peut généraliser à de nouvelles données.

« Les réseaux de neurones apprennent des exemples, mais ce qu'ils apprennent, ce sont des représentations. »

Bases de PyTorch

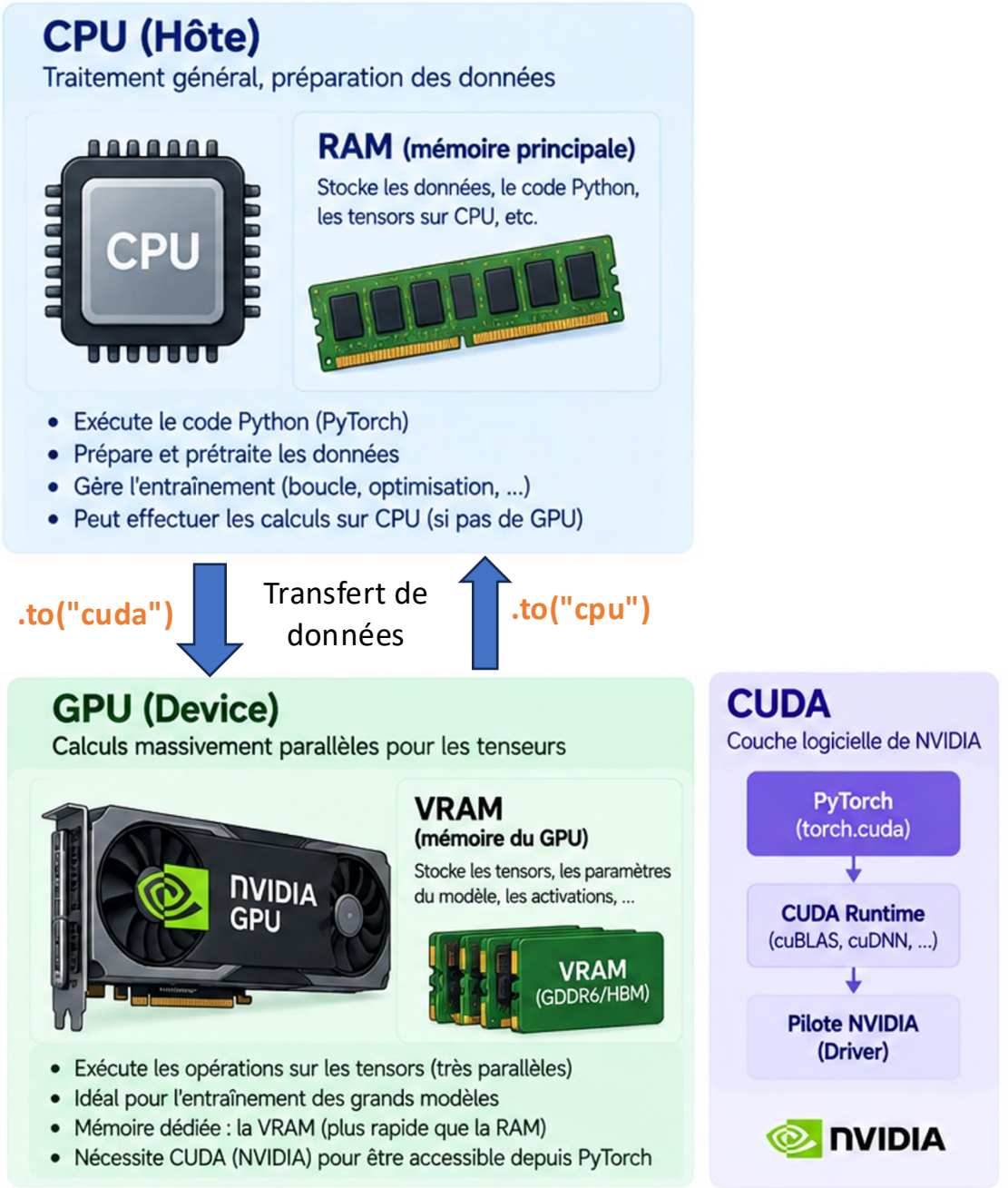
PyTorch est un framework open source de Machine Learning et de Deep Learning, principalement utilisé avec Python, qui permet de construire, entraîner et déployer des réseaux de neurones.

Préparation

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import matplotlib.pyplot as plt


print("PyTorch version:", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())
```

```
PyTorch version: 2.10.0
CUDA disponible: False
```



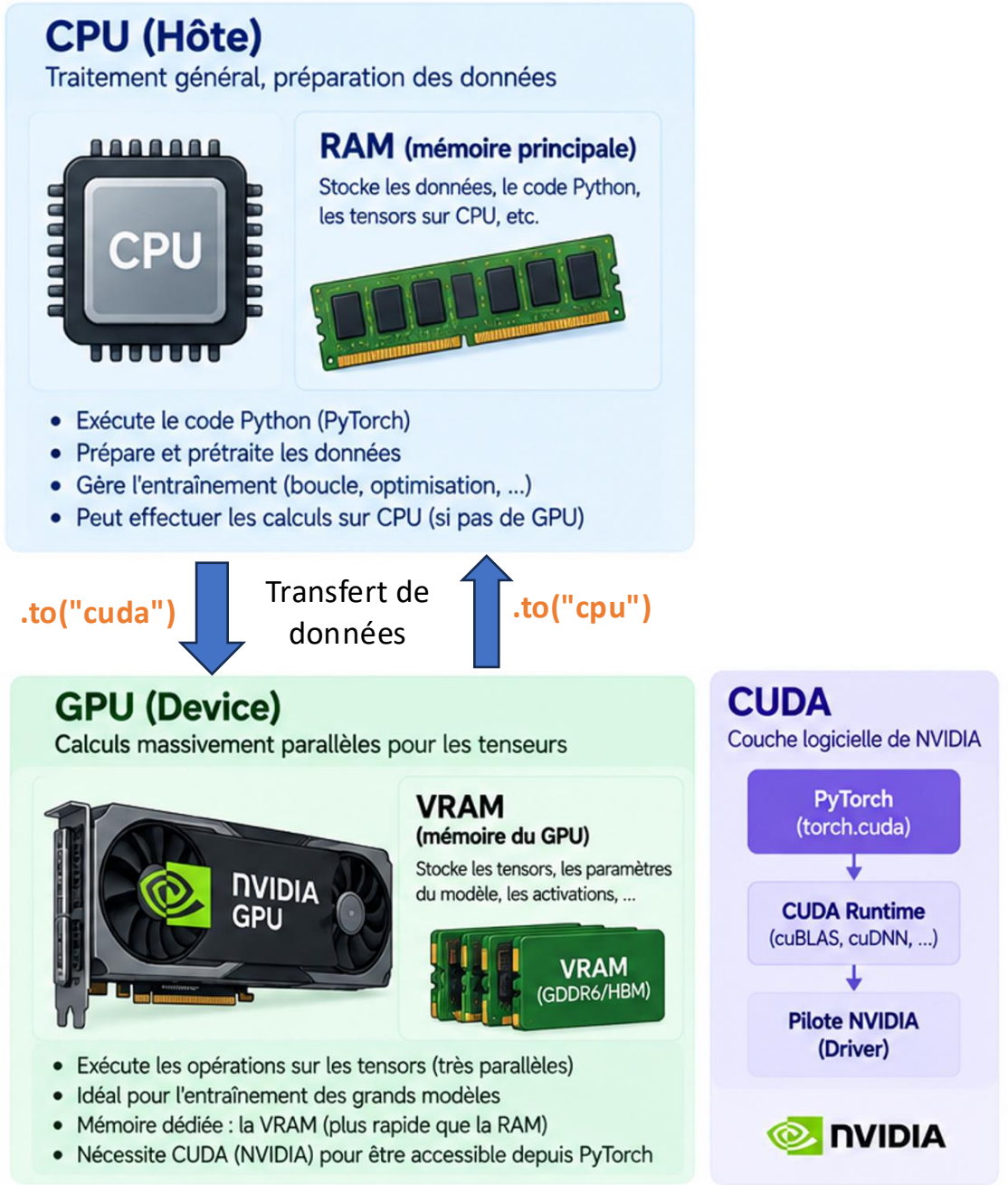
Bases de PyTorch

Tensors

```
scalar = torch.tensor(3.0)
vector = torch.tensor([1.0, 2.0, 3.0])
matrix = torch.tensor([[1.0, 2.0], [3.0, 4.0]])


print("scalar:", scalar, "shape =", scalar.shape)
print("vector:", vector, "shape =", vector.shape)
print("matrix:", matrix, "shape =", matrix.shape)
print("dtype:", matrix.dtype)
print("ndim:", matrix.ndim)
```

```
scalar: tensor(3.) shape = torch.Size([])
vector: tensor([1., 2., 3.]) shape = torch.Size([3])
matrix: tensor([[1., 2.],
        [3., 4.]]) shape = torch.Size([2, 2])
dtype: torch.float32
ndim: 2
```



Bases de PyTorch

Opération sur les tenseurs

```
x = torch.arange(12)
print("x =", x)

x2 = x.reshape(3, 4)
print("reshape(3,4):", x2)

print("Colonne 1:", x2[:, 1])
print("Ligne 0:", x2[0, :])

A = torch.tensor([[1., 2.], [3., 4.]])
B = torch.tensor([[2., 0.], [1., 2.]])
print("A @ B =", A @ B)

print("mean(A) =", A.mean())
print("sum(A) =", A.sum())
```

```
x = tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
reshape(3,4): tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
Colonne 1: tensor([1, 5, 9])
Ligne 0: tensor([0, 1, 2, 3])
A @ B = tensor([[ 4.,  4.],
        [10.,  8.]])
mean(A) = tensor(2.5000)
sum(A) = tensor(10.)
```

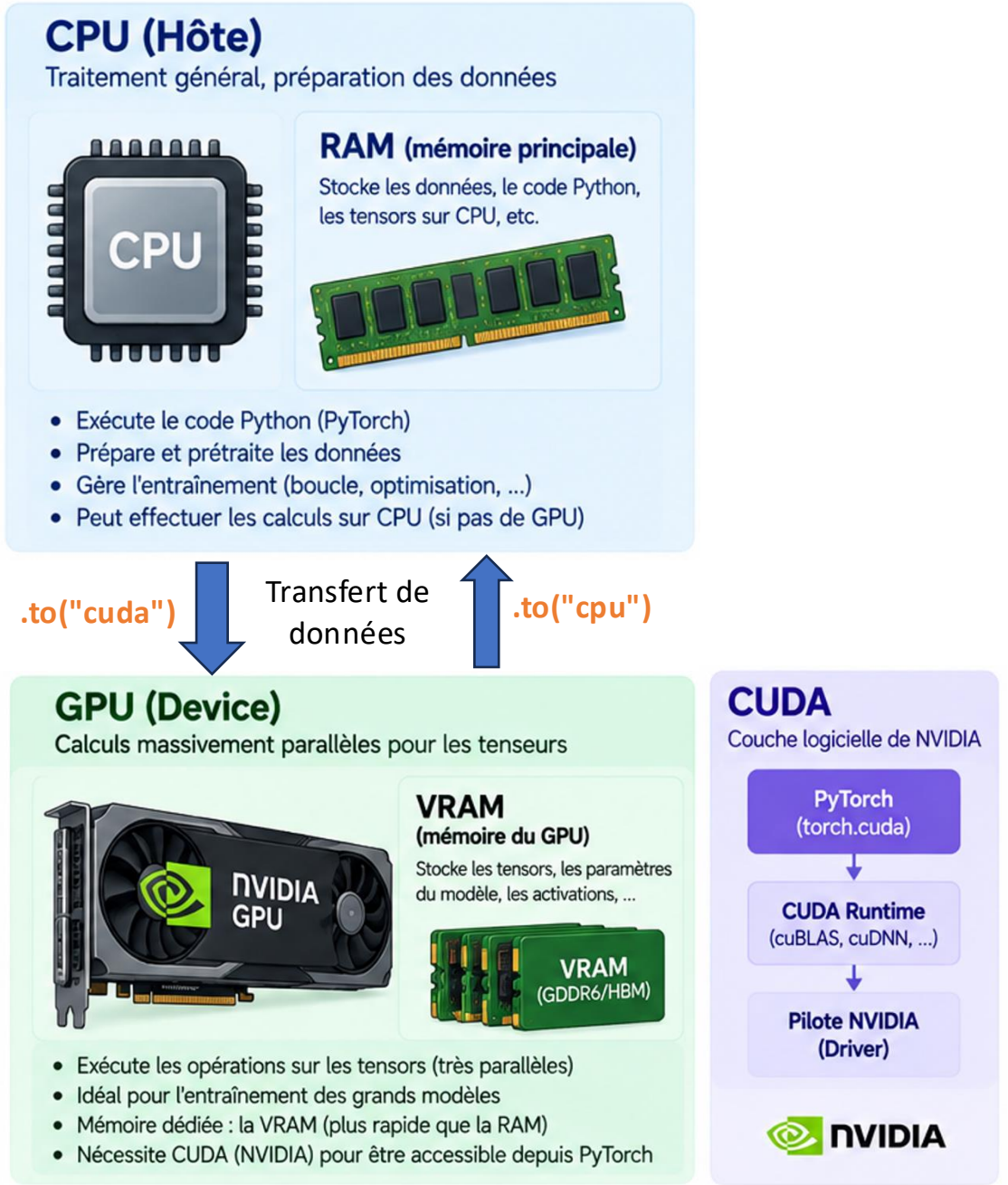
Bases de PyTorch

CPU, GPU et Devide

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device utilisé:", device)

x = torch.tensor([1.0, 2.0, 3.0]).to(device)
print("x.device =", x.device)
```

```
Device utilisé: cpu
x.device = cpu
```



Bases de PyTorch

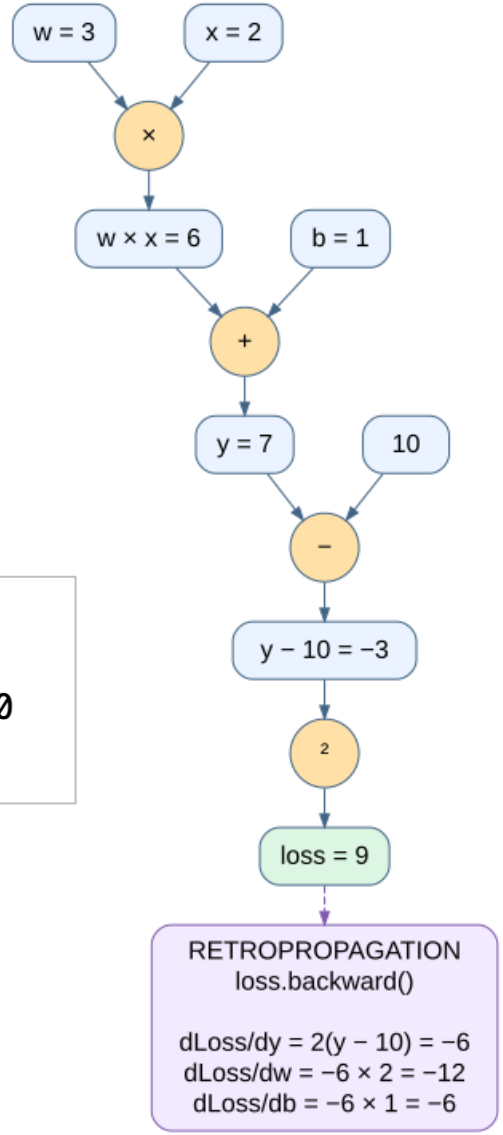
Autograd

```
x = torch.tensor(2.0)
w = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
y = w * x + b
loss = (y - 10) ** 2
loss.backward()
print("y =", y.item())
print("loss =", loss.item())
print("dLoss/dw =", w.grad.item())
print("dLoss/db =", b.grad.item())
```

```
y = 7.0
loss = 9.0
dLoss/dw = -12.0
dLoss/db = -6.0
```

```
// Une autre façonde faire
dw, db = torch.autograd.grad(loss, (w, b))
print("y =", y.item())
print("loss =", loss.item())
print("dLoss/dw =", dw.item())
print("dLoss/db =", db.item())
```

GRAPHE DES OPERATIONS



Vérification analytique

$$y = wx + b$$

$$L = (y - 10)^2$$

$$\frac{\partial L}{\partial w} = 2(y - 10)x$$

$$\frac{\partial L}{\partial b} = 2(y - 10)$$

```
y_val = (3.0 * 2.0 + 1.0)
dL_dw = 2 * (y_val - 10) * 2.0
dL_db = 2 * (y_val - 10)
print(dL_dw, dL_db)
```

```
-12.0 -6.0
```

Bases de PyTorch

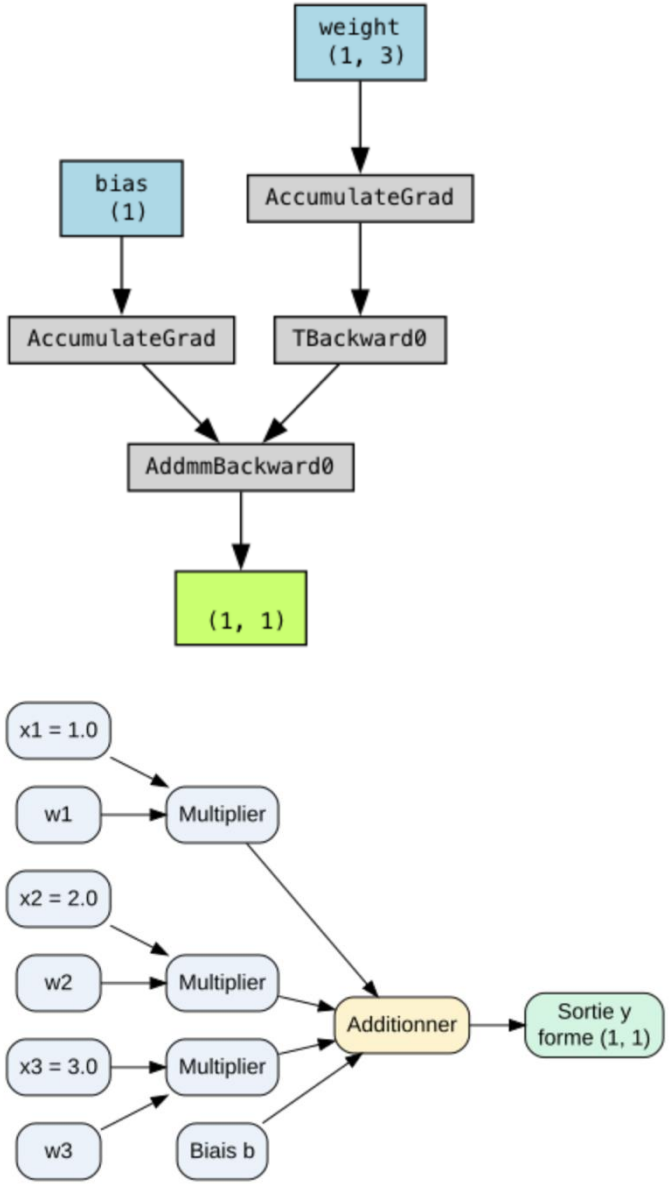
Un neuron avec nn.Linear

```
import torch
import torch.nn as nn
from torchviz import make_dot
model = nn.Linear(3, 1)
x = torch.tensor([[1.0, 2.0, 3.0]])
y = model(x)
print("Sortie:", y)
print("Poids:", model.weight)
print("Biais:", model.bias)
dot = make_dot(y, params=dict(model.named_parameters()))
dot.render("graphe_operations", format="png", cleanup=True)
```

$$z = \sum_i w_i x_i + b$$

```
Sortie: tensor([[1.0337]], grad_fn=<AddmmBackward0>)
Poids: Parameter containing:
tensor([[ 0.0909, -0.2441,  0.5697]], requires_grad=True)
Biais: Parameter containing:
tensor([-0.2783], requires_grad=True)

'graphe_operations.png'
```



Bases de PyTorch

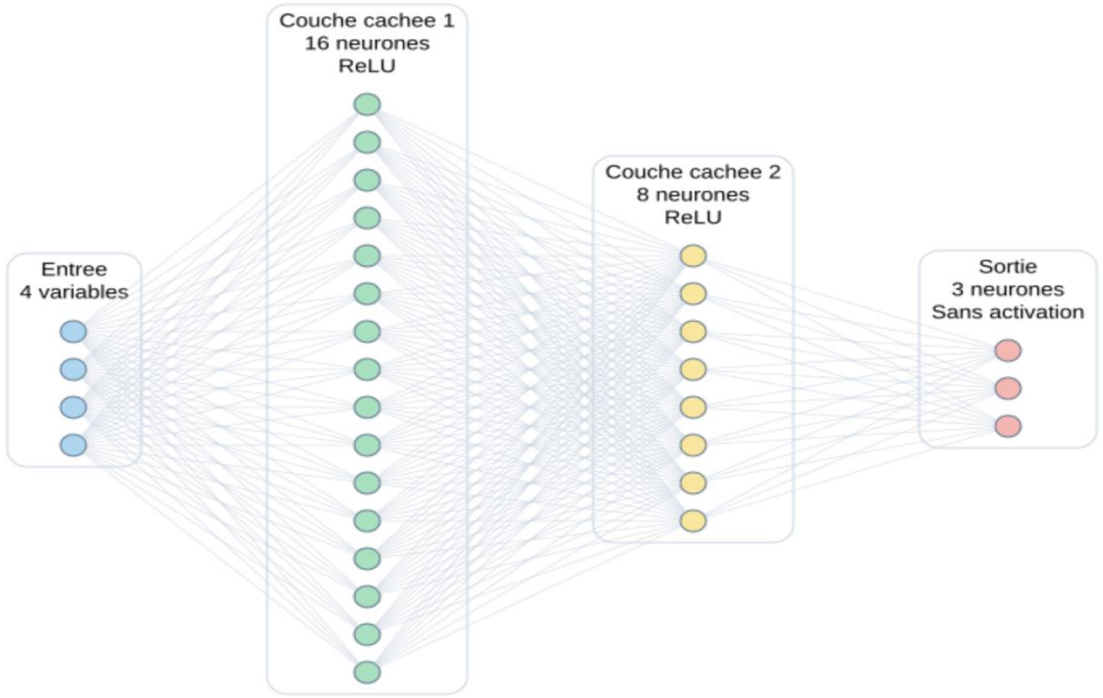
Construire un réseau de neurones

```
class MLP(nn.Module):
   def __init__(self):
      super().__init__()
      self.net = nn.Sequential(
         nn.Linear(4, 16),
         nn.ReLU(),
         nn.Linear(16, 8),
         nn.ReLU(),
         nn.Linear(8, 3)
      )

   def forward(self, x):
      return self.net(x)

model = MLP()
print(model)
```

```
MLP(
  (net): Sequential(
    (0): Linear(in_features=4, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=8, bias=True)
    (3): ReLU()
    (4): Linear(in_features=8, out_features=3, bias=True)
  )
)
```



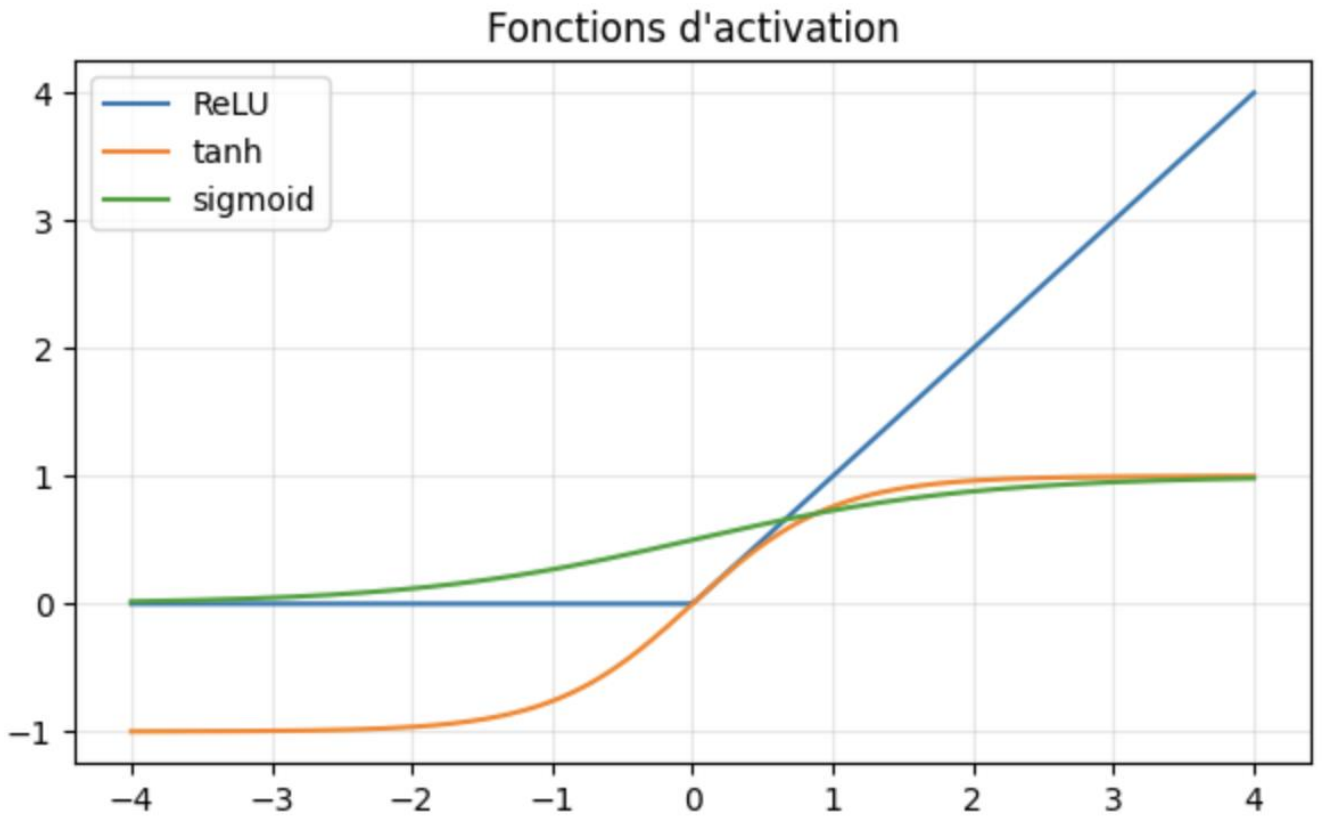

```
model = nn.Sequential(
         nn.Linear(4, 16),
         nn.ReLU(),
         nn.Linear(16, 8),
         nn.ReLU(),
         nn.Linear(8, 3)
      )
print(model)
```

Bases de PyTorch

Fonctions d'activation

```
x = torch.linspace(-4, 4, 200)
relu = torch.relu(x)
tanh = torch.tanh(x)
sigmoid = torch.sigmoid(x)


plt.figure(figsize=(7,4))
plt.plot(x.numpy(), relu.numpy(), label='ReLU')
plt.plot(x.numpy(), tanh.numpy(), label='tanh')
plt.plot(x.numpy(), sigmoid.numpy(), label='sigmoid')
plt.grid(True, alpha=0.3)
plt.legend()
plt.title("Fonctions d'activation")
plt.show()
```

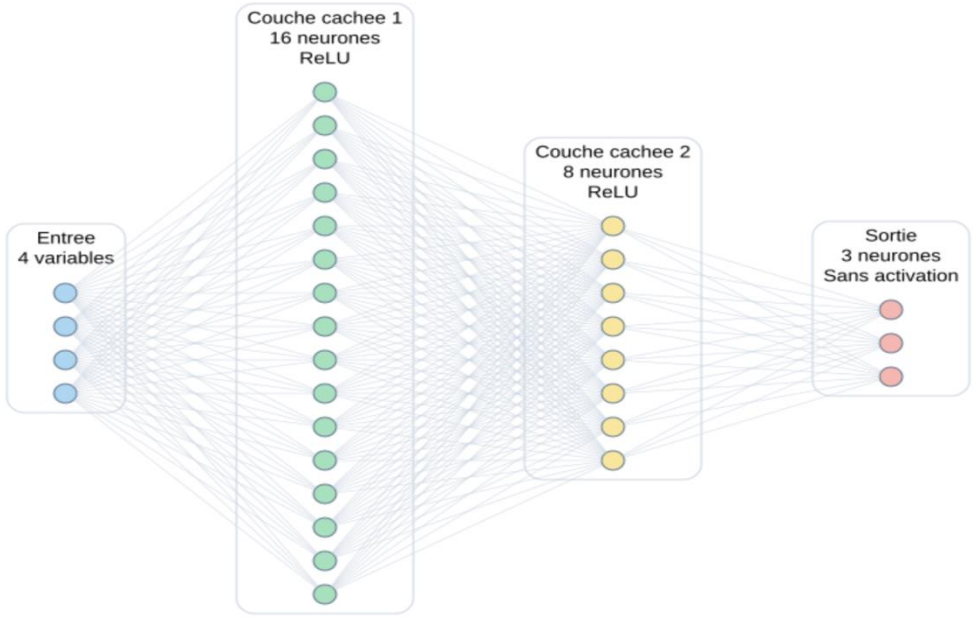


Bases de PyTorch

Forward Propagation

```
X = torch.randn(5, 4)
# L'appel model(X) déclenche automatiquement model.forward(X)
predicted = model(X)
print("Shape entrée :", X.shape)
print("Shape sortie :", predicted.shape)
```

```
Shape entrée : torch.Size([5, 4])
Shape sortie : torch.Size([5, 3])
```



Bases de PyTorch

Loss Function : Fonction coût

```
# Régression
criterion_reg = nn.MSELoss()
y_pred = torch.tensor([[2.0], [4.0]])
y_true = torch.tensor([[3.0], [5.0]])
print("MSE =", criterion_reg(y_pred, y_true).item())

# Classification multi-classe
criterion_cls = nn.CrossEntropyLoss()
pred = torch.tensor([[2.0, 1.0, 0.5], [0.2, 0.1, 2.0]])
y = torch.tensor([0, 2])
print("CrossEntropy =", criterion_cls(pred, y).item())
```

Cas principaux :

- régression : `nn.MSELoss()` ;
- classification multi-classe : `nn.CrossEntropyLoss()` ;
- classification binaire : `nn.BCEWithLogitsLoss()`.

```
MSE = 1.0
CrossEntropy = 0.36905235052108765
```

Bases de PyTorch

L'optimiseur utilise les gradients pour mettre à jour les paramètres.

Optimizers

```
simple_model = nn.Linear(1, 1)
optimizer = torch.optim.Adam(simple_model.parameters(), lr=1e-2)
print(optimizer)
```

```
Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    decoupled_weight_decay: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    weight_decay: 0
)
```

Bases de PyTorch

Boucle d'apprentissage

```
X = torch.randn(8, 1)
y = 2 * X + 1
simple_model = nn.Linear(1, 1)
optimizer = torch.optim.Adam(simple_model.parameters(), lr=1e-2)
criterion = nn.MSELoss()
epochs = 500
# Exemple générique
for epoch in range(epochs):
    simple_model.train()
    y_pred = simple_model(X)
    loss = criterion(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 100 ==0 :
        print(f"epoch={epoch} loss={loss.item():.4f}")
```

```
epoch=0 loss=9.4138
epoch=100 loss=3.2036
epoch=200 loss=0.8631
epoch=300 loss=0.1912
epoch=400 loss=0.0346
```

Bases de PyTorch

train() vs eval()

```
class DropoutDemo(nn.Module):
    def __init__(self):
        super().__init__()
        self.dropout = nn.Dropout(p=0.5)

    def forward(self, x):
        return self.dropout(x)

m = DropoutDemo()
x = torch.ones(10)

m.train()
print("train() :", m(x))

m.eval()
with torch.inference_mode():
    print("eval()  :", m(x))
```

```
train() : tensor([0., 2., 0., 2., 2., 0., 0., 0., 2., 0.])
eval()  : tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

train() et eval() configurent le comportement du modèle ;

- model.train() et model.eval() influencent notamment le comportement de **Dropout** et **BatchNorm**.
- `eval()` ne désactive pas le calcul des gradients. Pour l'inférence, on utilise généralement `torch.inference_mode()`.

Le dropout sert à **limiter le surapprentissage**, en empêchant le réseau de dépendre systématiquement des mêmes activations.

En mode train(), cette couche :

- met chaque valeur à **0 avec une probabilité de 50 %** ;
- multiplie les valeurs conservées par (1/(1-0{,}5)=2), pour préserver leur moyenne en espérance.

On obtient donc des **0 et des 2**, répartis aléatoirement :

En mode eval(), Le dropout est désactivé : il laisse passer les valeurs sans les modifier. Le résultat est donc :

Bases de PyTorch

DataSet et DataLoader

```
X = torch.randn(100, 3)
y = torch.randn(100, 1)

dataset = TensorDataset(X, y)
loader = DataLoader(dataset, batch_size=16, shuffle=True)

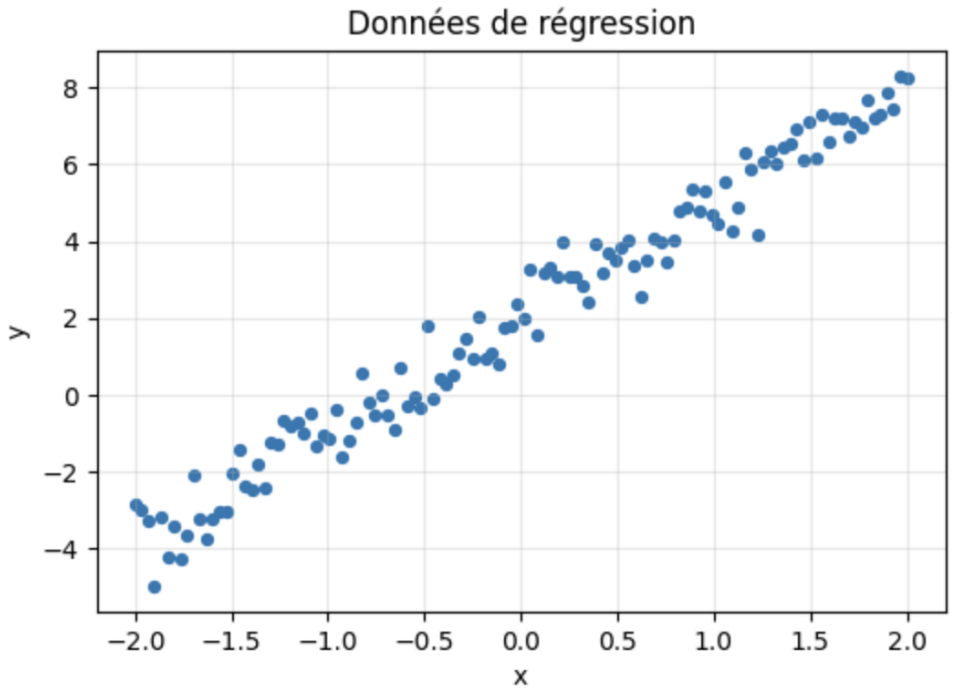
for X_batch, y_batch in loader:
    print("Batch X:", X_batch.shape, "Batch y:", y_batch.shape)
    break
```

```
Batch X: torch.Size([16, 3]) Batch y: torch.Size([16, 1])
```

Bases de PyTorch

Régression linéaire : y = 3x + 2 + bruit

```
torch.manual_seed(42)
X = torch.linspace(-2, 2, 120).unsqueeze(1)
noise = 0.6 * torch.randn_like(X)
y = 3 * X + 2 + noise
plt.figure(figsize=(6,4))
plt.scatter(X.numpy(), y.numpy(), s=18)
plt.title("Données de régression")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, alpha=0.3)
plt.show()
```



Bases de PyTorch

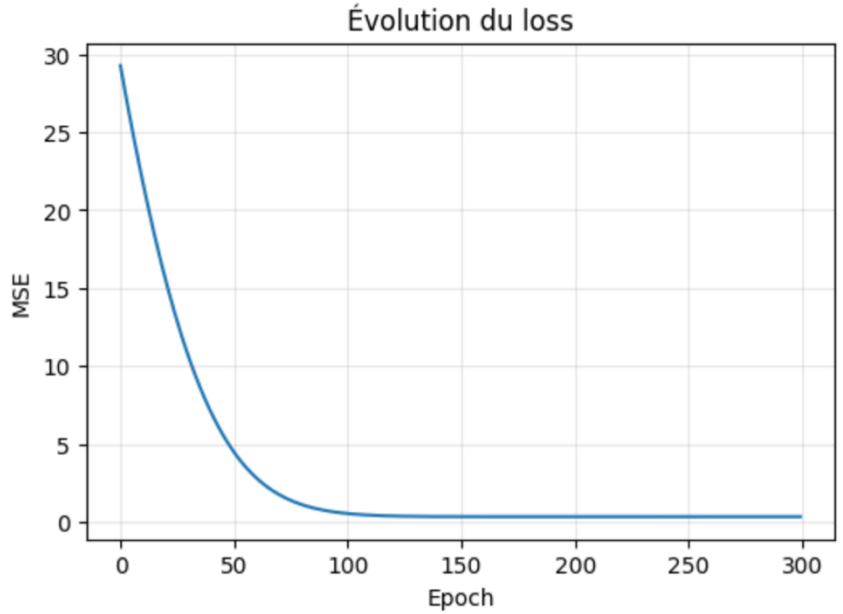
Régression linéaire : y = 3x + 2 + bruit

```
model_reg = nn.Linear(1, 1)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model_reg.parameters(), lr=0.05)
```

```
losses = []
for epoch in range(300):
    model_reg.train()
    y_pred = model_reg(X)
    loss = criterion(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
print("Dernier loss:", losses[-1])
print("w appris:", model_reg.weight.item())
print("b appris:", model_reg.bias.item())
```

```
Dernier loss: 0.3438418209552765
w appris: 2.9894580841064453
b appris: 2.030014753341675
```

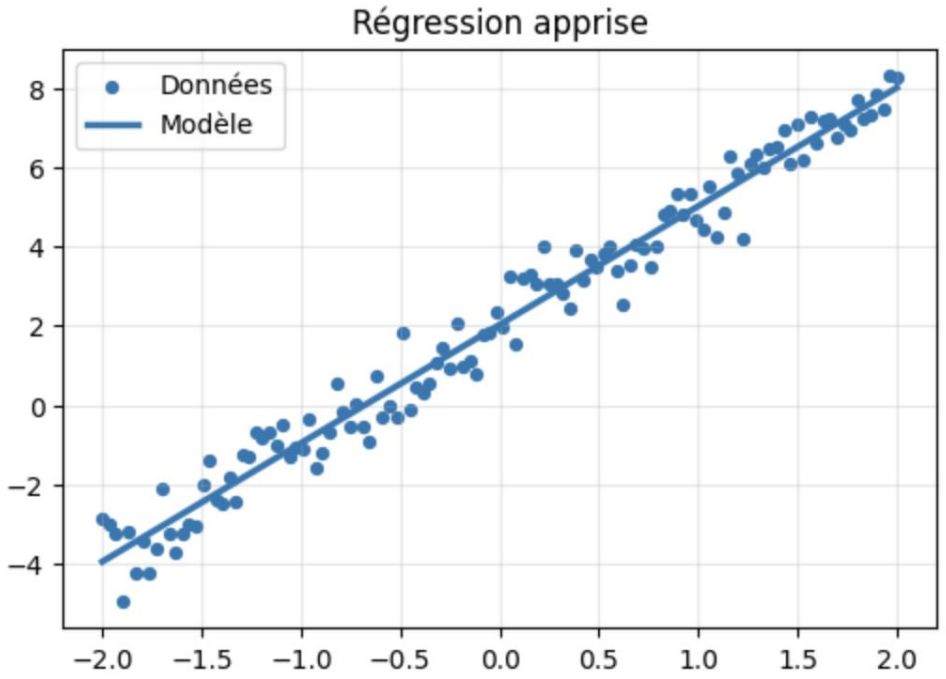
```
plt.figure(figsize=(6,4))
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.title("Évolution du loss")
plt.grid(True, alpha=0.3)
plt.show()
```



Bases de PyTorch

Régression linéaire : y = 3x + 2 + bruit

```
model_reg.eval()
with torch.inference_mode():
   y_hat = model_reg(X)
plt.figure(figsize=(6,4))
plt.scatter(X.numpy(), y.numpy(), s=18, label="Données")
plt.plot(X.numpy(), y_hat.numpy(), linewidth=2.5, label="Modèle")
plt.legend()
plt.title("Régression apprise")
plt.grid(True, alpha=0.3)
plt.show()
```



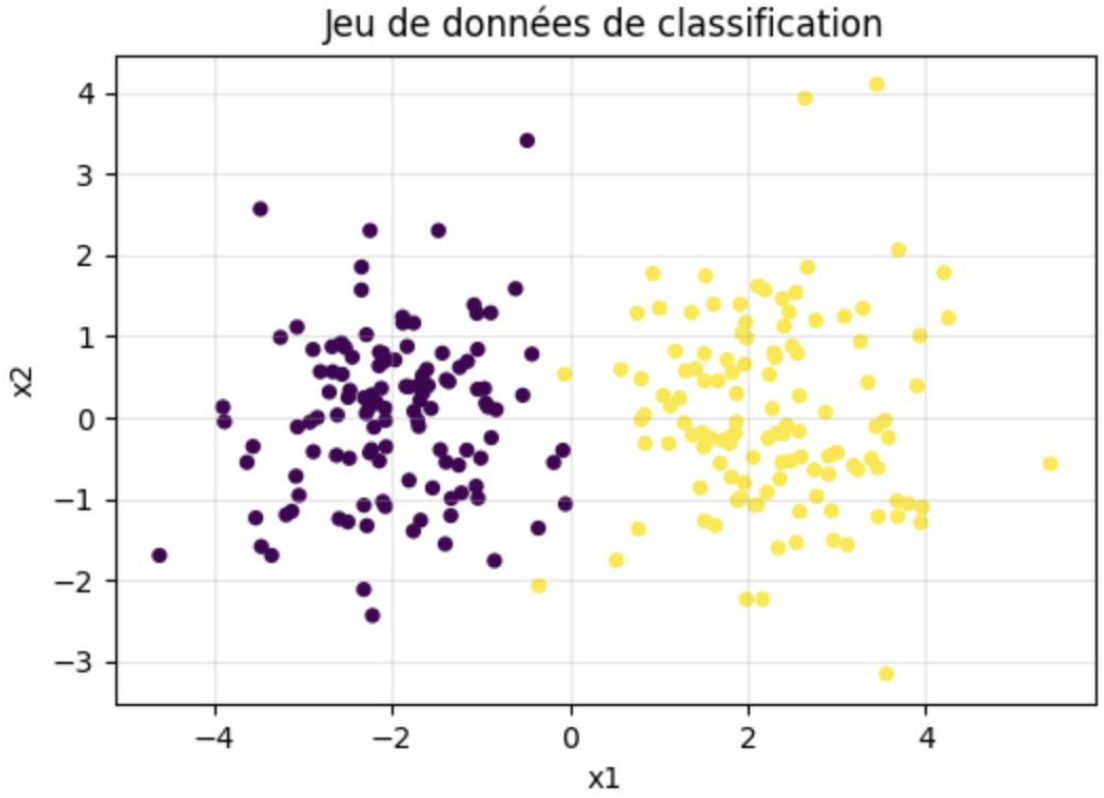

```
from torcheval.metrics import R2Score
metric = R2Score()
metric.update(y_hat,y)
r2=metric.compute()
print(f"R2 Score = {r2:.3f}")
```

R2 Score = 0.972

Bases de PyTorch

Classification : Création de deux classes de points en 2D.

```
torch.manual_seed(0)
n = 120
class0 = torch.randn(n, 2) + torch.tensor([-2.0, 0.0])
class1 = torch.randn(n, 2) + torch.tensor([ 2.0, 0.0])
X_cls = torch.cat([class0, class1], dim=0)
y_cls = torch.cat([
    torch.zeros(n, dtype=torch.long),
    torch.ones(n, dtype=torch.long)
])
plt.figure(figsize=(6,4))
plt.scatter(X_cls[:,0], X_cls[:,1], c=y_cls, s=18)
plt.title("Jeu de données de classification")
plt.xlabel("x1")
plt.ylabel("x2")
plt.grid(True, alpha=0.3)
plt.show()
```



Bases de PyTorch

Classification : Modèle

```
class Classifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 16),
            nn.ReLU(),
            nn.Linear(16, 16),
            nn.ReLU(),
            nn.Linear(16, 2)
        )
    def forward(self, x):
        return self.net(x)
model_cls = Classifier()
criterion_cls = nn.CrossEntropyLoss()
optimizer_cls = torch.optim.Adam(model_cls.parameters(), lr=0.01)
```

```
losses_cls = []
for epoch in range(250):
    model_cls.train()
    logits = model_cls(X_cls)
    loss = criterion_cls(logits, y_cls)
    optimizer_cls.zero_grad()
    loss.backward()
    optimizer_cls.step()
    losses_cls.append(loss.item())
print("Dernier loss:", losses_cls[-1])
```

```
Dernier loss: 0.0006943692569620907
```

```
model_cls.eval()
with torch.inference_mode():
    logits = model_cls(X_cls)
    pred = logits.argmax(dim=1)
    accuracy = (pred == y_cls).float().mean()
print(f"Accuracy = {accuracy.item()*100:.2f}%")
```

```
Accuracy = 100.00%
```

Bases de PyTorch

Classification : Métriques d'évaluation

```
from torcheval.metrics import BinaryAccuracy, BinaryRecall, BinaryPrecision, BinaryPrecision, BinaryF1Score
from torchmetrics.classification import MulticlassConfusionMatrix, BinaryConfusionMatrix
```

```
accuracy_metric = BinaryAccuracy()
accuracy_metric.update(y_cls,pred)
accuracy_value = accuracy_metric.compute()
```

```
recall_metric = BinaryRecall()
recall_metric.update(y_cls, pred)
recall_value = recall_metric.compute()
```

```
precision_metric = BinaryPrecision()
precision_metric.update(y_cls,pred)
precision_value = precision_metric.compute()
```

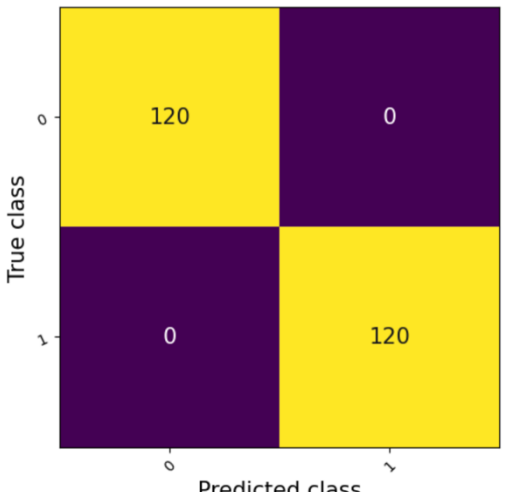
```
f1_score_metric = BinaryF1Score()
f1_score_metric.update(y_cls, pred)
f1_score_value = f1_score_metric.compute()
```

```
print(f"Accuracy = {accuracy_value:.4f}, recall= {recall_value:.4f}, precision={precision_value:.4f},
f1score={f1_score_value:.4f}")
```

```
Accuracy = 1.0000, recall= 1.0000, precision=1.0000, f1score=1.0000
```

```
metric = BinaryConfusionMatrix()
confusion_matrix = metric(pred, y_cls)
print(confusion_matrix)
metric.plot()
```

```
tensor([[120,   0],
        [  0, 120]])
```

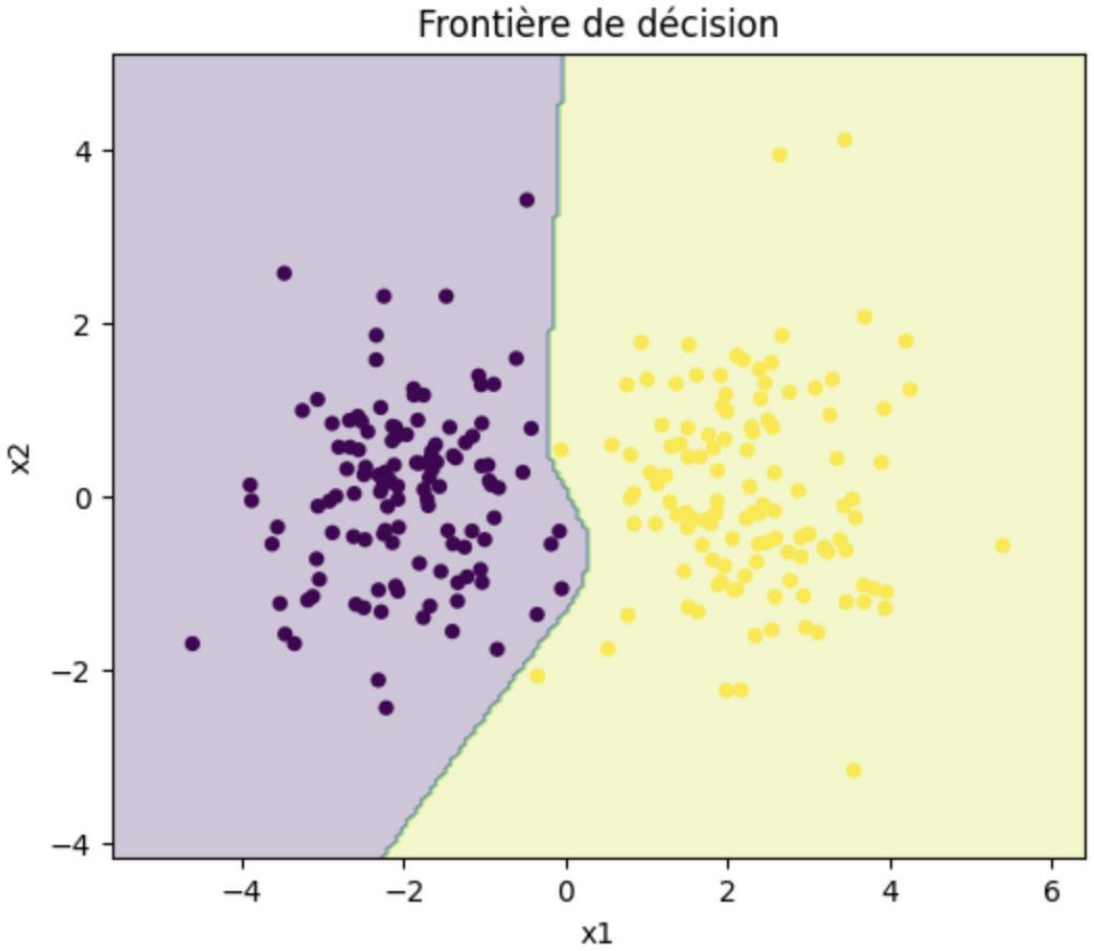


Bases de PyTorch

Classification : Visualisation de la frontière de décision

```
x1_min, x1_max = X_cls[:,0].min()-1, X_cls[:,0].max()+1
x2_min, x2_max = X_cls[:,1].min()-1, X_cls[:,1].max()+1
xx1, xx2 = torch.meshgrid(
    torch.linspace(x1_min, x1_max, 200),
    torch.linspace(x2_min, x2_max, 200),
    indexing='ij'
)
grid = torch.stack([xx1.reshape(-1), xx2.reshape(-1)], dim=1)
```

```
model_cls.eval()
with torch.inference_mode():
    zz = model_cls(grid).argmax(dim=1).reshape(xx1.shape)
plt.figure(figsize=(6,5))
plt.contourf(xx1.numpy(), xx2.numpy(), zz.numpy(), alpha=0.25)
plt.scatter(X_cls[:,0], X_cls[:,1], c=y_cls, s=18)
plt.title("Frontière de décision")
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()
```



Bases de PyTorch

Classification : Sauvegarde et recharge du modèle

```
path = "classifier_state_dict.pt"
torch.save(model_cls.state_dict(), path)
print("Modèle sauvegardé dans", path)


new_model = Classifier()
new_model.load_state_dict(torch.load(path, weights_only=True))
new_model.eval()

with torch.inference_mode():
   pred_new = new_model(X_cls).argmax(dim=1)

print("Prédictions identiques:", torch.equal(pred, pred_new))
```

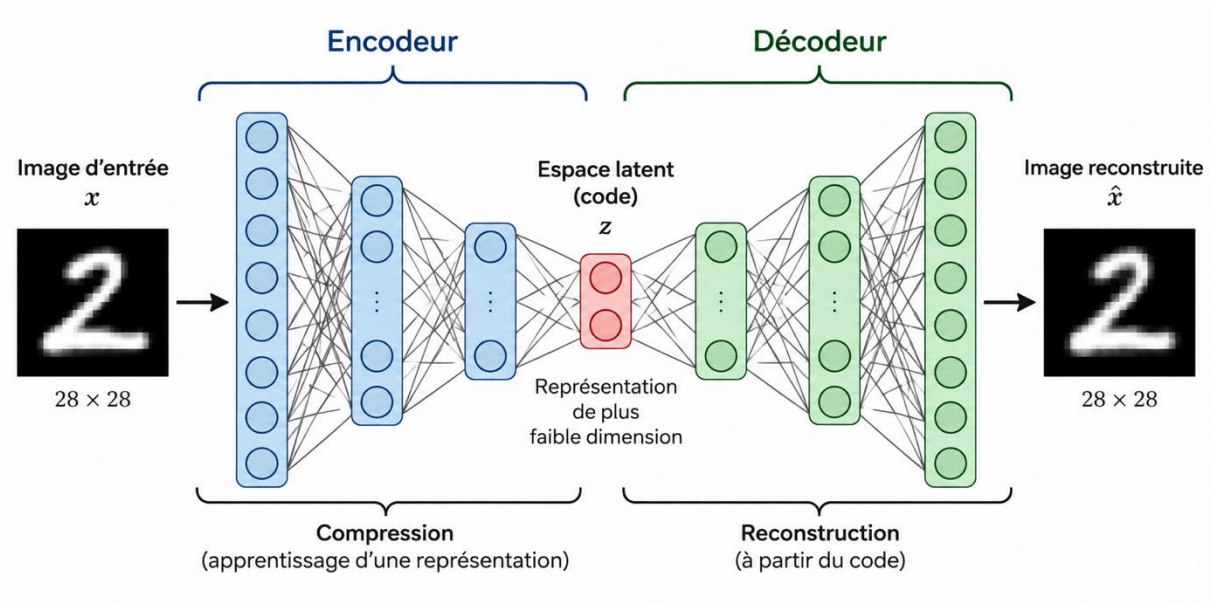
```
Modèle sauvegardé dans classifier_state_dict.pt
Prédictions identiques: True
```

Introduction aux modèles génératifs

- Auto encodeurs : Reconstruction et représentation latente.
- Auto encodeurs variationnels : VAE.
- Réseaux antagonistes génératifs : GAN.
- Modèles de diffusion.
- Introduction aux modèles autorégressifs.
- Mise ne œuvre d’un réseau de classification et un autoencodeur simple avec PyTorch.

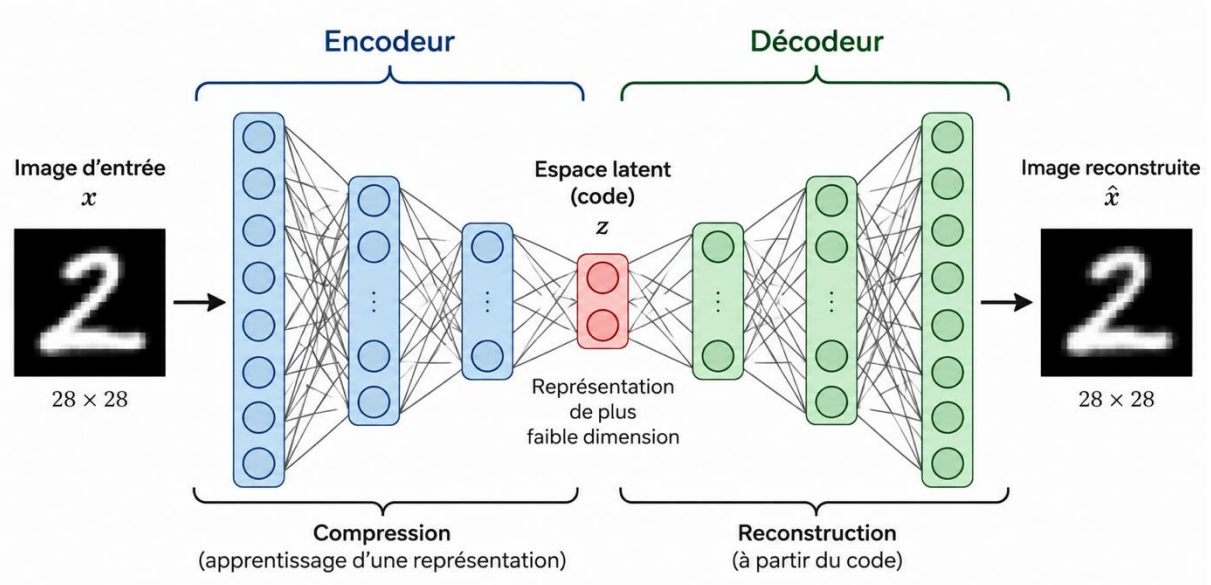
Auto-Encodeur

- La reconstruction est un objectif d'apprentissage. Elle oblige le modèle, lorsqu'on lui impose des contraintes adaptées, à conserver les informations utiles dans une représentation interne.
- L'objectif n'est donc pas uniquement de copier une entrée, mais d'apprendre une représentation exploitable des données.
- Question : Si vous deviez décrire une image avec seulement quelques nombres, quelles informations faudrait-il conserver ?



Auto-Encodeur

- Si vous deviez décrire une image avec seulement quelques nombres, quelles informations faudrait-il conserver ?
  - conserver les informations essentielles qui permettent de reconnaître l'image et d'en reconstruire une approximation : la forme générale des objets, leurs contours, leur position, leur orientation et, si elles sont importantes, leurs couleurs. Les détails fins ou le bruit pourraient être laissés de côté. »
  - Exemple d'un chiffre manuscrit : quelques nombres pourraient décrire la présence d'une boucle, l'inclinaison, la courbure et l'épaisseur des traits, plutôt que mémoriser chaque pixel.
  - c'est le rôle de la **représentation latente** : encoder les informations utiles à la reconstruction dans un petit nombre de valeurs. Le réseau apprend lui-même ce codage ; chaque valeur ne correspond pas nécessairement à une caractéristique identifiable.



Architecture et Apprentissage

$x \in R^d$

L'encodeur calcule :

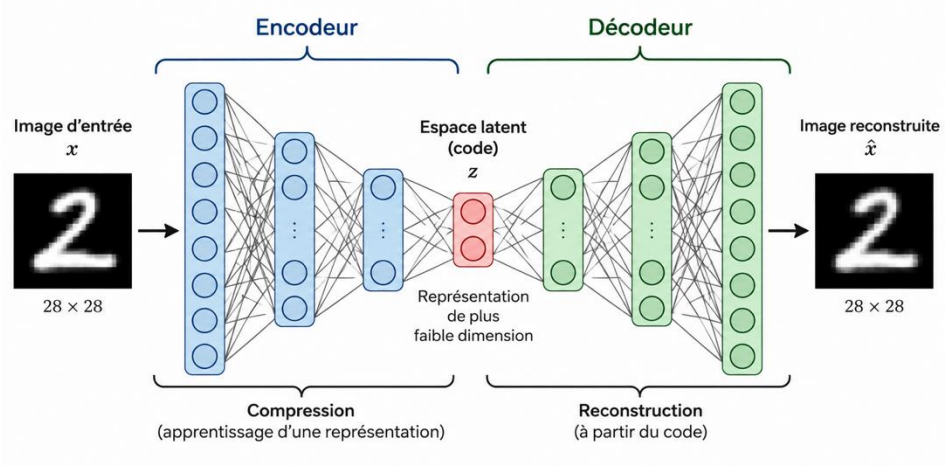
$z = f_\theta(x), \quad z \in R^k$

Le décodeur produit :

$\hat{x} = g_\phi(z)$

Ainsi :

$\hat{x} = g_\phi(f_\theta(x))$



- Θ représente les paramètres de l'encodeur ;
- Ø représente ceux du décodeur ;
- (z) est le **code latent** ;
- $\hat{x}$ est la reconstruction.
- Lorsque (k<d), on parle d'un auto-encodeur **sous-complet** : la représentation latente constitue un goulot d'étranglement.

L'entraînement minimise l'écart entre l'entrée et sa reconstruction :

$$[\min_{\theta,\phi} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\text{rec}\,(x_i, g\phi(f_\theta(x_i)))]$$

Erreur quadratique moyenne — MSE : Pour des données continues :

$$\mathrm{L_{MSE}}(x, \hat{x}) \frac{1}{d} \sum_{j=1}^{d} (x_j - \hat{x}_j)^2$$

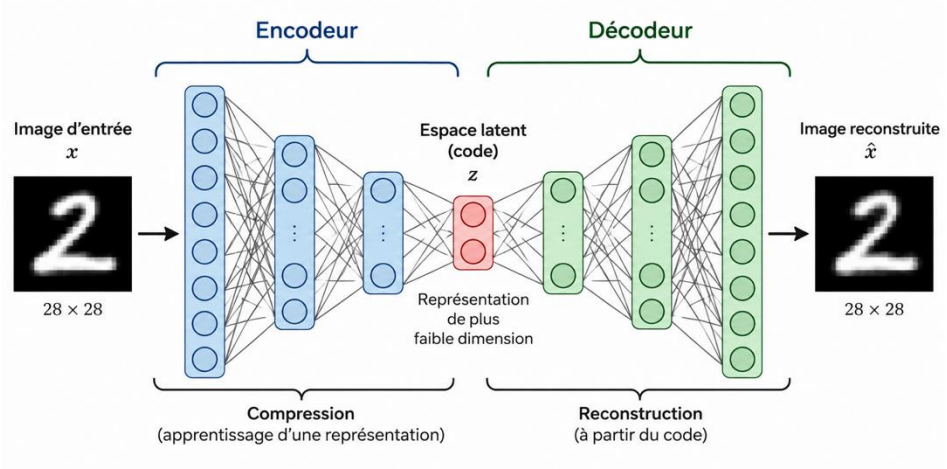
Entropie croisée binaire — BCE : Pour des données binaires ou une modélisation des composantes par des lois de Bernoulli :

$$\mathrm{L_{BCE}}(x, \hat{x}) - \frac{1}{d} \sum_{j=1}^{d} [x_j \log(\hat{x}_j) + (1 - x_j) \log(1 - \hat{x}_j)]$$

Étapes d’entraînement

1. Sélectionner un lot de données.
2. Calculer les représentations latentes.
3. Reconstruire les entrées.
4. Calculer la perte de reconstruction.
5. Rétro-propager les gradients.
6. Mettre à jour les paramètres.

$\hat{x}$



L’entraînement minimise l’écart entre l’entrée et sa reconstruction :

$$[\ \min_{\theta,\phi} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\mathrm{rec}\,(x_i, g\phi(f_\theta(x_i)))]$$

Erreur quadratique moyenne — MSE : Pour des données continues :

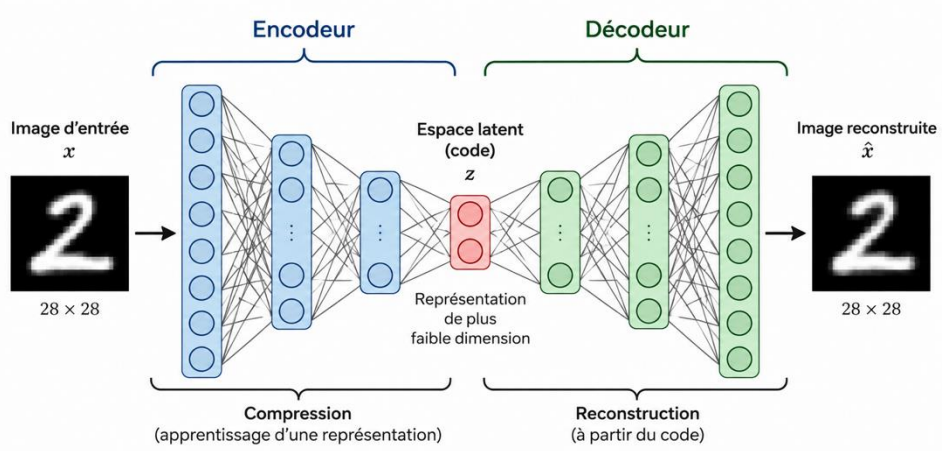
$$\mathrm{L}_{\mathrm{MSE}}(x, \hat{x}) \frac{1}{d} \sum_{j=1}^{d} (x_j - \hat{x}_j)^2$$

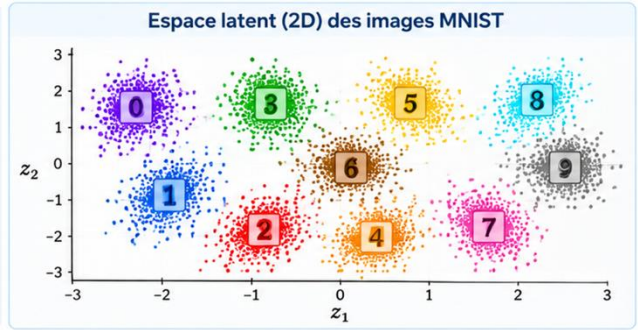
Entropie croisée binaire — BCE : Pour des données binaires ou une modélisation des composantes par des lois de Bernoulli :

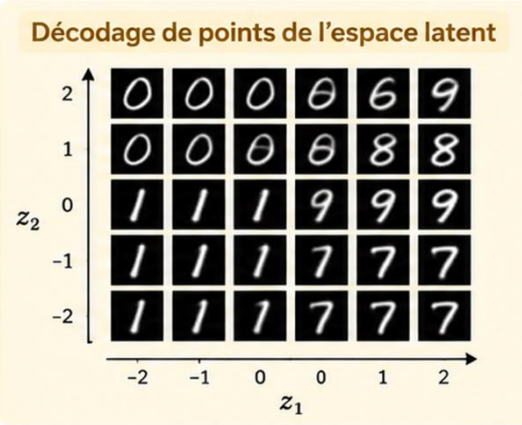
$$\mathrm{L}_{\mathrm{BCE}}(x, \hat{x}) - \frac{1}{d} \sum_{j=1}^{d} [x_j \log(\hat{x}_j) + (1 - x_j) \log(1 - \hat{x}_j)]$$

Comprendre la représentation latente

- **Que contient le code latent ?**
  - Le vecteur (z) représente les informations que le modèle utilise pour reconstruire l'entrée.
  - Pour des chiffres manuscrits, il peut notamment refléter :
    - l'inclinaison ;
    - l'épaisseur des traits ;
    - la courbure ;
    - la forme générale du chiffre.
  - Cependant, **rien ne garantit qu'une coordonnée latente corresponde à une caractéristique interprétable**. L'information peut être répartie sur plusieurs dimensions.





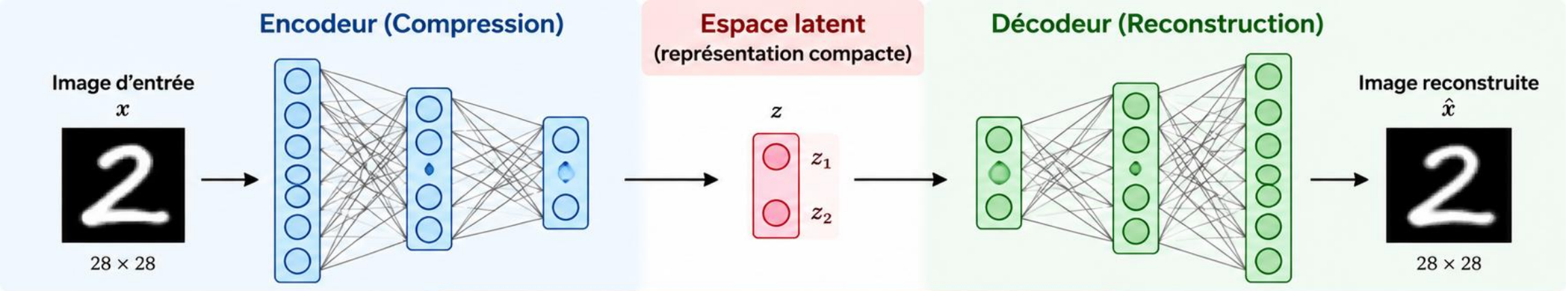


Influence de la dimension latente

Dimension latente	Effet possible
Trop faible	Perte d'informations et reconstruction dégradée
Intermédiaire	Compromis entre compression et fidélité
Très élevée	Reconstruction facilitée, mais représentation parfois peu structurée

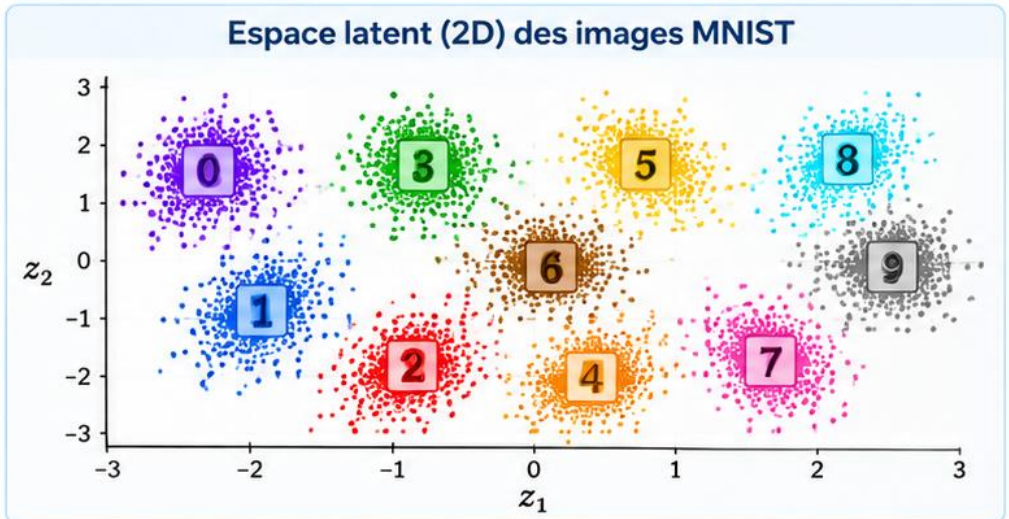
Une grande capacité peut permettre au réseau d'apprendre une fonction proche de l'identité, voire de mémoriser les données d'entraînement.

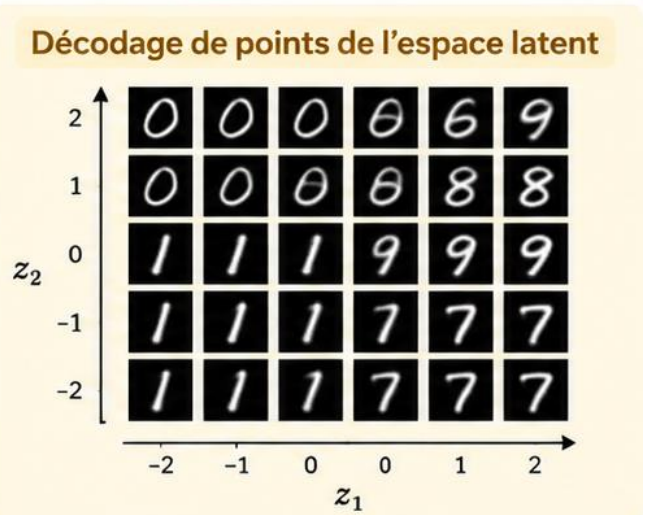
Comprendre la représentation latente



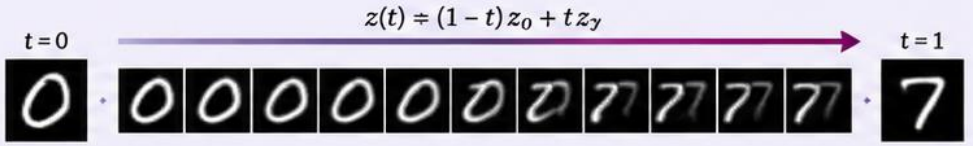
Que représente l'espace latent ?

- Chaque image est encodée en un **vecteur z de faible dimension** (ici 2D pour la visualisation).
- Des **images similaires** se retrouvent **proches** dans cet espace.
- Chaque **point** de l'espace latent correspond à une image **« plausible »** (qui ressemble à un chiffre MNIST).
- Le décodeur transforme tout **point z** de cet espace en une image.





Interpolation dans l'espace latent



- En se déplaçant progressivement dans l'espace latent entre deux points (par exemple un « 0 » et un « 7 »), on obtient des **images intermédiaires cohérentes**.

Variantes principales

Variante	Principe	Intérêt
Sous-complet	Limiter la dimension latente	Réduction de dimension
Débruitant	Reconstruire une donnée propre à partir d'une entrée corrompue	Robustesse au bruit
Parcimonieux	Pénaliser certaines activations latentes	Représentations peu actives
Convolutionnel	Utiliser des convolutions	Exploiter la structure spatiale des images
Variationnel — VAE	Apprendre une distribution latente régularisée	Génération par échantillonnage

Applications

- **Réduction de dimension** et visualisation.
- **Extraction de caractéristiques** pour une tâche ultérieure.
- **Débruitage** d'images ou de signaux.
- **Détection d'anomalies** par erreur de reconstruction.
- **Compression**, en ajoutant les mécanismes de quantification et de codage nécessaires.

Limites :

- Une anomalie peut parfois être bien reconstruite.
- Une faible MSE ne garantit pas une bonne qualité perceptuelle.
- L'espace latent n'est pas nécessairement interprétable.
- Les performances dépendent du prétraitement et de la capacité du réseau.
- La reconstruction sur les données d'entraînement ne suffit pas à évaluer le modèle.

Implémentation d'un auto-encodeur

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader

from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

print("PyTorch :", torch.__version__)

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
print("Device :", device)
```

- https://ossci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz

```
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

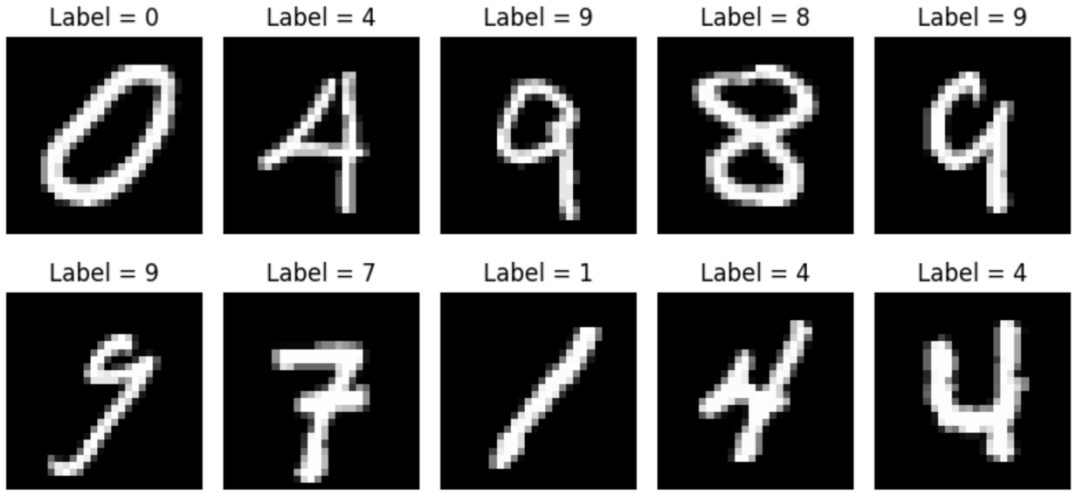
print("Train :", len(train_dataset))
print("Test  :", len(test_dataset))
```

```
data
└── mnist
    └── raw
        ├── t10k-images-idx3-ubyte
        ├── t10k-images-idx3-ubyte.gz
        ├── t10k-labels-idx1-ubyte
        ├── t10k-labels-idx1-ubyte.gz
        ├── train-images-idx3-ubyte
        ├── train-images-idx3-ubyte.gz
        ├── train-labels-idx1-ubyte
        └── train-labels-idx1-ubyte.gz
```

Implémentation d’un auto-encodeur

```
images, labels = next(iter(train_loader))

fig, axes = plt.subplots(2, 5, figsize=(8, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(images[i].squeeze(), cmap="gray")
    ax.set_title(f"Label = {labels[i].item()}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```



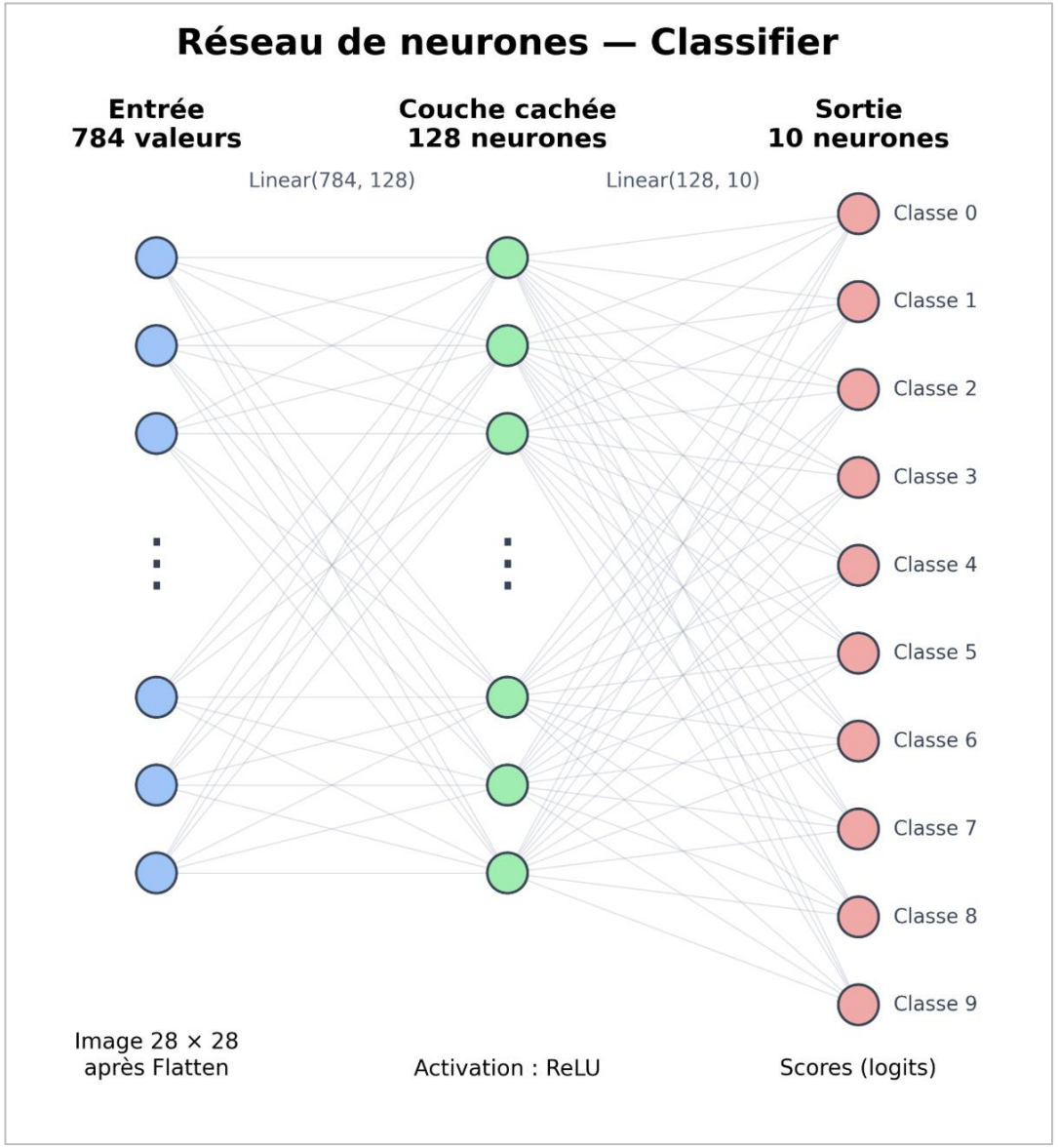
Modèle discriminatif

```
class Classifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.net(x)

classifier = Classifier().to(device)
classifier
```

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(classifier.parameters(), lr=1e-3)
```



Modèle discriminatif

```python
def train_classifier(model, loader, epochs=3):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for x, y in loader:
            x, y = x.to(device), y.to(device)

            logits = model(x)
            loss = criterion(logits, y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch+1}/{epochs} - "
            f"Loss = {total_loss/len(loader):.4f}"
        )
train_classifier(classifier, train_loader, epochs=3)
```

```
Epoch 1/3 - Loss = 0.4073
Epoch 2/3 - Loss = 0.1934
Epoch 3/3 - Loss = 0.1408
```

```python
def evaluate_classifier(model, loader):
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)

            logits = model(x)
            pred = logits.argmax(dim=1)

            correct += (pred == y).sum().item()
            total += y.size(0)

    return correct / total

accuracy = evaluate_classifier(classifier, test_loader)
print(f"Accuracy = {100*accuracy:.2f}%")
```

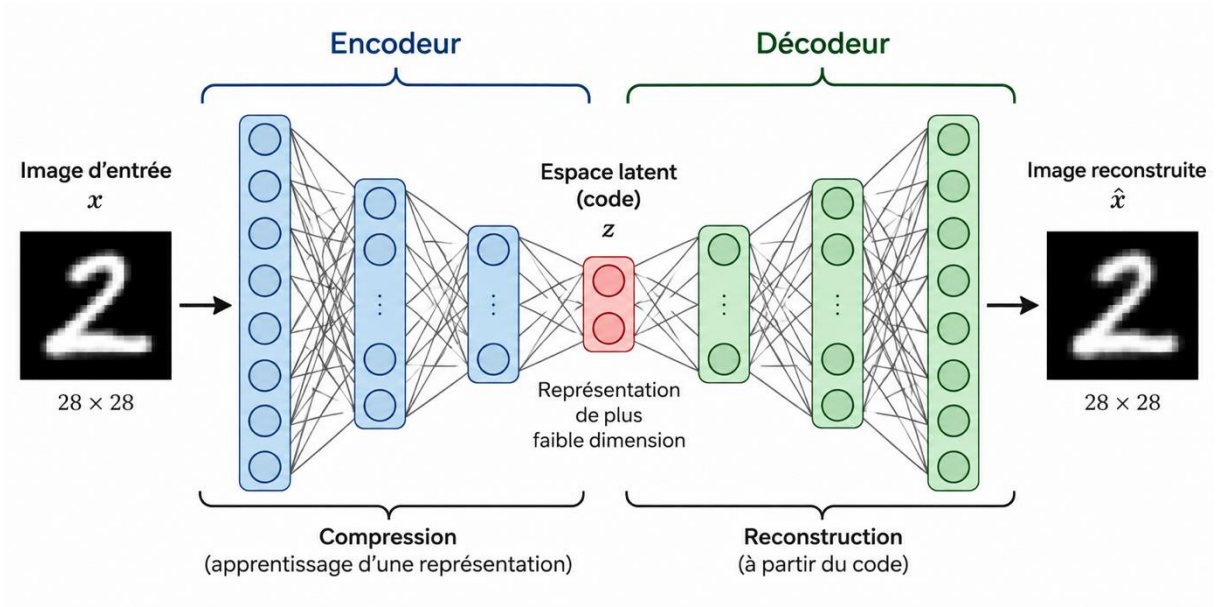
```
Accuracy = 96.41%
```

Modèle Génératif : Auto-encodeur

```
class Autoencoder(nn.Module):
    def __init__(self, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 32),
            nn.ReLU(),
            nn.Linear(32, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
            nn.Sigmoid()
        )
    def encode(self, x):
        return self.encoder(x)

    def decode(self, z):
        x_hat = self.decoder(z)
        return x_hat.view(-1, 1, 28, 28)

    def forward(self, x):
        z = self.encode(x)
        x_hat = self.decode(z)
        return x_hat
ae = Autoencoder(latent_dim=2).to(device)
ae
```




```
Autoencoder(
  (encoder): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=32, bias=True)
    (4): ReLU()
    (5): Linear(in_features=32, out_features=2, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=2, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=784, bias=True)
    (5): Sigmoid()
  )
)
```

Modèle Génératif : Auto-encodeur

```
ae_criterion = nn.MSELoss()
ae_optimizer = torch.optim.Adam(ae.parameters(), lr=1e-3)
```

```
def train_autoencoder(model, loader, epochs=5):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for x, _ in loader:
            x = x.to(device)

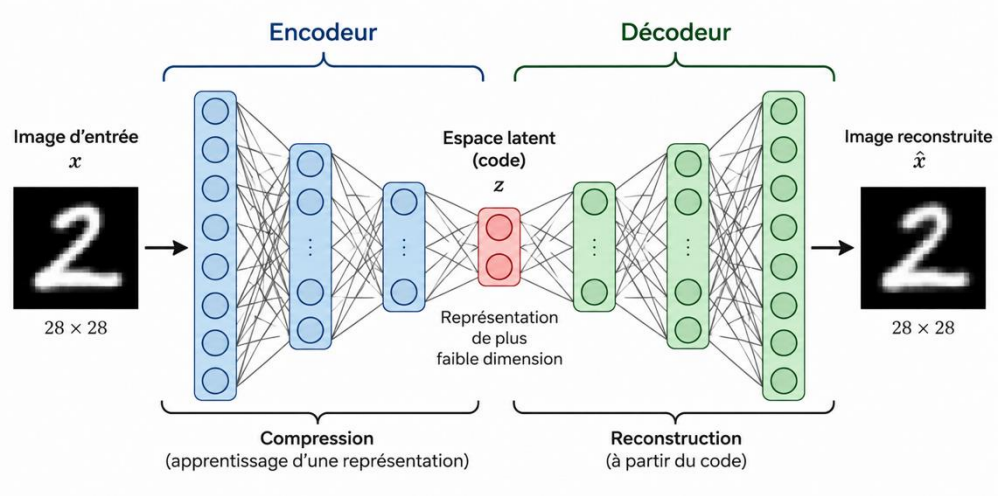
            x_hat = model(x)
            loss = ae_criterion(x_hat, x)

            ae_optimizer.zero_grad()
            loss.backward()
            ae_optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch+1}/{epochs} - "
            f"Loss = {total_loss/len(loader):.4f}"
        )

train_autoencoder(ae, train_loader, epochs=5)
```




```
Epoch 1/5 - Loss = 0.0657
Epoch 2/5 - Loss = 0.0511
Epoch 3/5 - Loss = 0.0477
Epoch 4/5 - Loss = 0.0457
Epoch 5/5 - Loss = 0.0446
```

Modèle Génératif : Auto-encodeur

```
ae.eval()

x, _ = next(iter(test_loader))
x = x[:10].to(device)

with torch.no_grad():
    x_hat = ae(x)

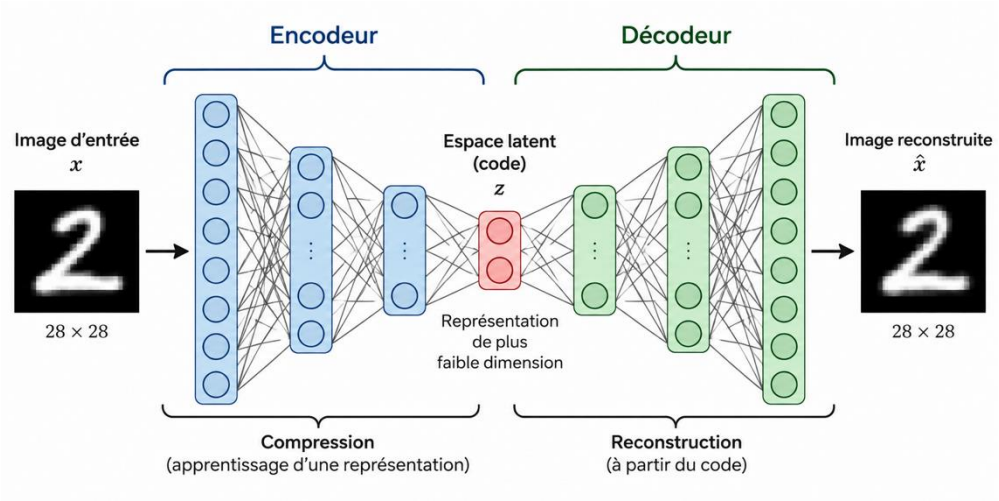
fig, axes = plt.subplots(2, 10, figsize=(15, 3))

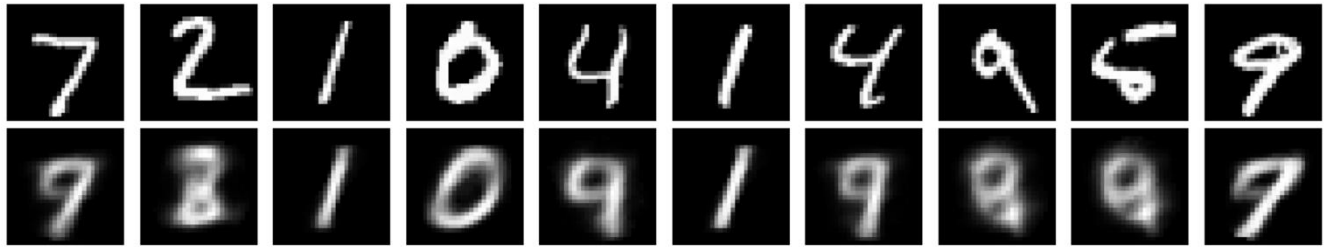
for i in range(10):
    axes[0, i].imshow(x[i].cpu().squeeze(), cmap="gray")
    axes[0, i].axis("off")

    axes[1, i].imshow(x_hat[i].cpu().squeeze(), cmap="gray")
    axes[1, i].axis("off")

axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("Reconstruit")

plt.tight_layout()
plt.show()
```





Modèle Génératif : Auto-encodeur

```
ae.eval()

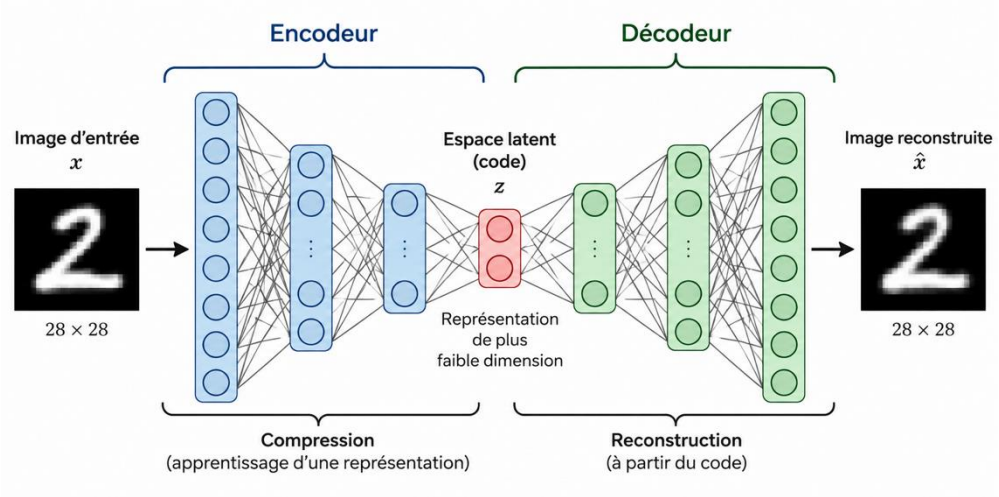
zs = []
ys = []

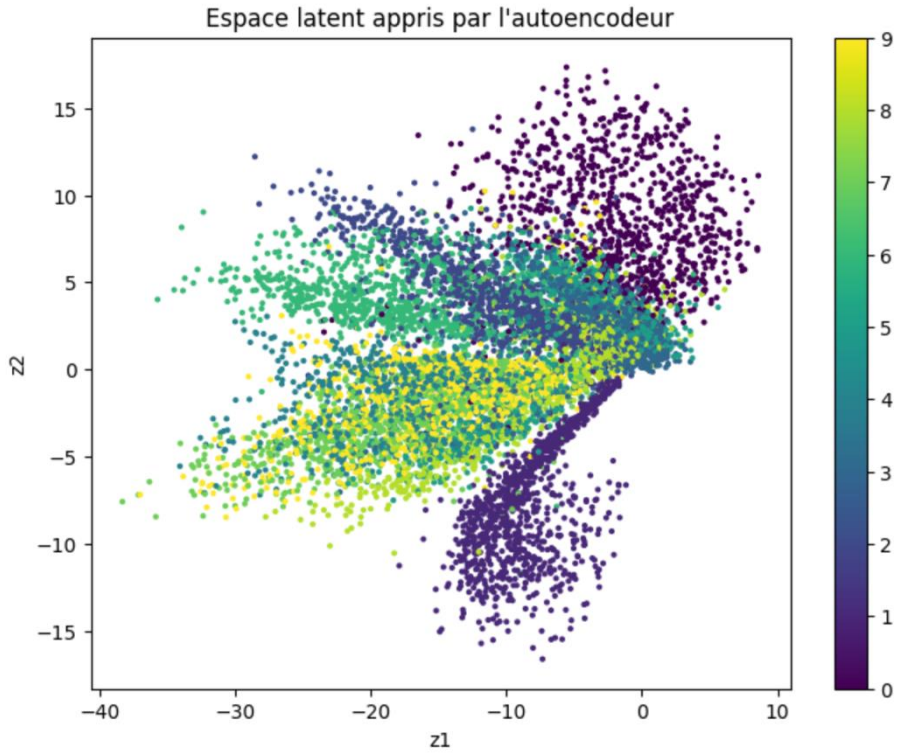
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        z = ae.encode(x)

        zs.append(z.cpu())
        ys.append(y)

Z = torch.cat(zs).numpy()
Y = torch.cat(ys).numpy()

plt.figure(figsize=(8, 6))
scatter = plt.scatter(Z[:, 0], Z[:, 1], c=Y, s=4)
plt.xlabel("z1")
plt.ylabel("z2")
plt.title("Espace latent appris par l'autoencodeur")
plt.colorbar(scatter)
plt.show()
```



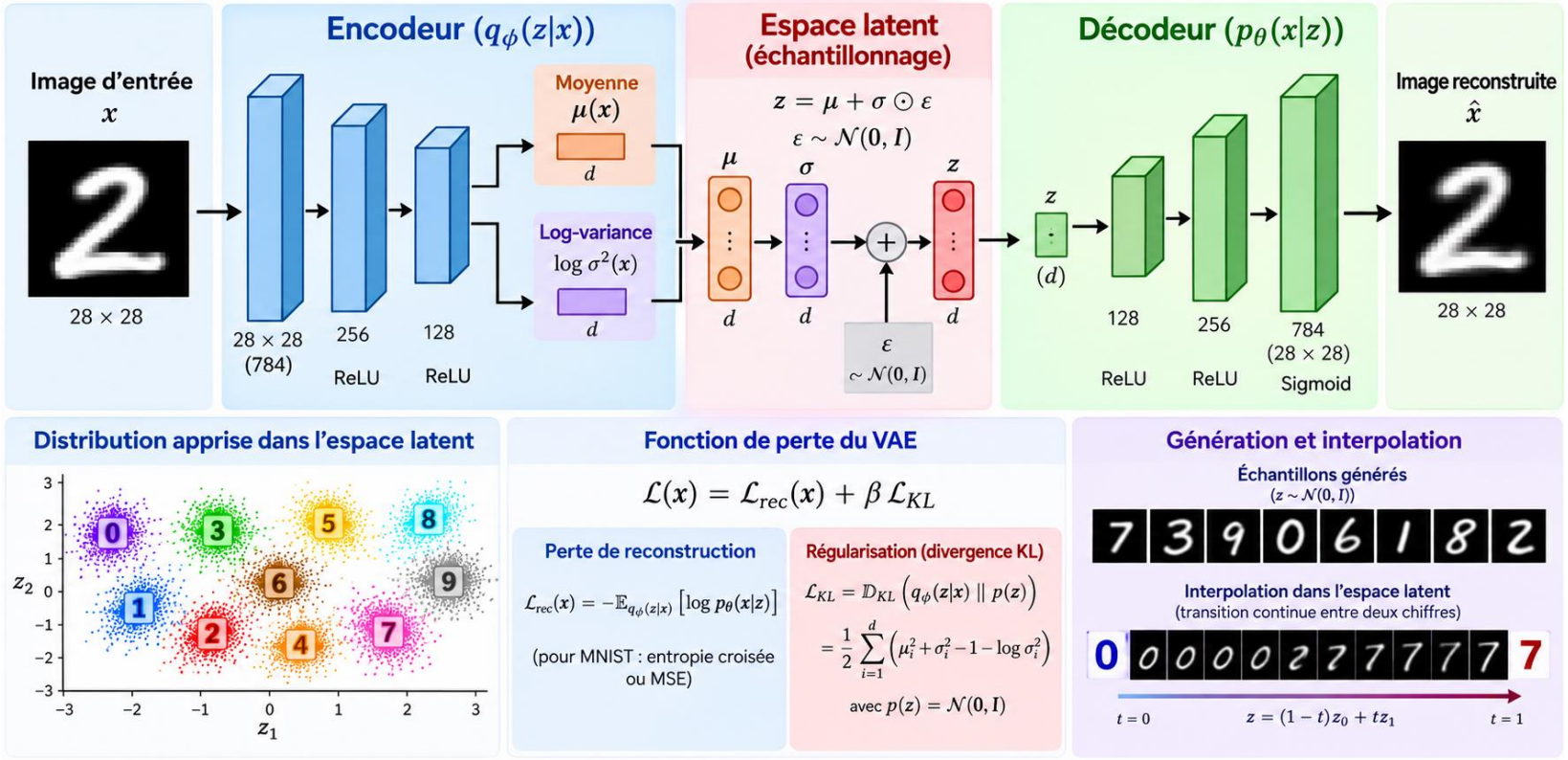


Modèle Génératif : Variational Auto Encoder (VAE)

Le **Variational Autoencoder**, ou **autoencodeur variationnel**, introduit une approche probabiliste :

- l'encodeur produit une **distribution latente** pour chaque entrée ;
- on échantillonne un code dans cette distribution ;
- le décodeur utilise ce code pour modéliser les données ;
- une régularisation rapproche les distributions latentes d'une distribution de référence.

Cette organisation permet de générer de nouvelles données en échantillonnant dans la distribution de référence.

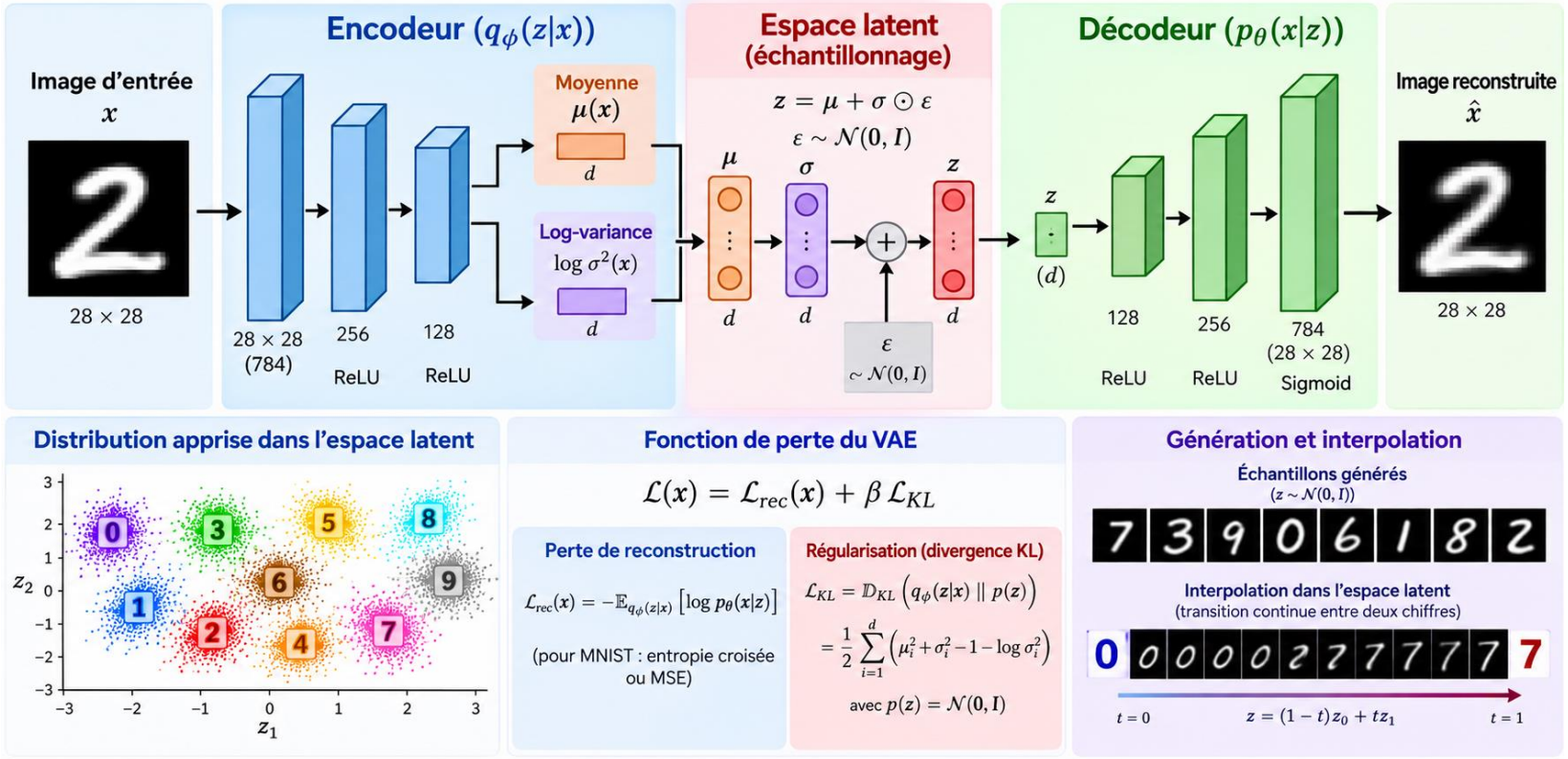


Modèle Génératif : Variational Auto Encoder (VAE)

Le **Variational Autoencoder**, ou **autoencodeur variationnel**, introduit une approche probabiliste :

- l'encodeur produit une **distribution latente** pour chaque entrée ;
- on échantillonne un code dans cette distribution ;
- le décodeur utilise ce code pour modéliser les données ;
- une régularisation rapproche les distributions latentes d'une distribution de référence.

Cette organisation permet de générer de nouvelles données en échantillonnant dans la distribution de référence.



Au lieu de dire « cette image correspond exactement à ce point », l'encodeur dit « cette image correspond plutôt à cette zone de l'espace latent ».